

22 JULY 2025

Quick Work

Page No.:

Date:

M T W T F S S

FORMAL LANGUAGE AND AUTOMATA THEORY

Formal language -

- 1) A set of strings formed from a finite set of symbols (alphabet) based on specific rules (grammar)
- 2) It is used to define programming languages and computational problems clearly and unambiguously.
- 3) A formal language is typically described by:
 - (i) An alphabet (a finite ~~set~~ set of symbols) ^{alphabet}
 - (ii) A set of strings (finite sequence of symbols from the
 - (iii) Sometimes a grammar or set of rules that determines which strings are valid
- 4) Classified into various types based on their complexity and computational models required to recognize them.
- 5) Classification is structured in the Chomsky hierarchy, which includes:
 - (i) Regular languages
 - (ii) Context free languages
 - (iii) Context sensitive languages
 - (iv) Recursively ~~no~~ enumerable languages

Basic Terminology: Alphabet -

- 1) An alphabet is a finite, non empty set of symbols
 - 2) We use the symbol Σ (sigma) to denote an alphabet
- eg: Binary: $\Sigma = \{0, 1\}$
All lower case letters: $\Sigma = \{a, b, c, \dots, z\}$
Alphanumeric: $\Sigma = \{a-z, A-Z, 0-9\}$
DNA molecule letters: $\Sigma = \{a, c, g, t\}$

Basic Terminology: Strings -

- 1) A string is a list of symbols from an alphabet (Σ), and the empty string is denoted by ϵ .
- 2) The length of a string is the number of symbols in it, denoted by $|w|$.
- 3) Concatenation joins 2 strings, represented as xy (eg. if $w_1 = abc$ and $w_2 = aab$, then $w_1 w_2 = abcaab$)
- 4) Kleene Star (Σ^*) represents the set of all possible strings over Σ , including the empty string.
- 5) The reverse of a string is formed by writing it backward (eg. reverse of "abc" is "cba")

Powers of an alphabet -

Let Σ be an alphabet $\Sigma = \{a, b\}$

- (i) Σ^k = set of all strings of length k
- (ii) $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots = \{\epsilon, a, b, aa, ab, ba, bb, \dots\}$
- (iii) $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots = \Sigma^* - \epsilon$

Languages & Grammar -

1) Languages:

A language is a collection of sentences of finite length all constructed from a finite alphabet of symbols

2) Grammar:

A grammar can be regarded as a device that enumerates the sentences of a language

$\Rightarrow$ An alphabet is a set of symbols:

$\{0, 1\}$

$\Rightarrow$ Words/Sentences are strings of symbols:

$0, 100, 01, 10, \dots$

$\Rightarrow$ A grammar language is a set of sentences:

$L = \{000, 0100, 0010, \dots\}$

$\Rightarrow$ A grammar is a finite list of rules defining a language

Language
 $S \rightarrow OA$
 $A \rightarrow 1A$
 $A \rightarrow 0B$
 $B \rightarrow 1B$
 $B \rightarrow 0F$
 $F \rightarrow \epsilon$

$\Rightarrow$ L is said to be a language over alphabet Σ only if $L \subseteq \Sigma^*$

$\Rightarrow$ This is because Σ^* is the set of all strings (of all possible length including ϵ) over the given alphabet Σ .

eg. (i) Let L be the language of all strings consisting of n 0's followed by n 1's:

$L = \{\epsilon, 01, 0011, 000111, \dots\}$

(ii) Let L be the language of all strings with equal number of 0's and 1's:

$L = \{\epsilon, 01, 10, 0011, 1100, 0101, 1010, 1001, \dots\}$

concerned ordering of strings in language

Definition: ϕ denotes the empty language

Let $L = \{\epsilon\}$, is $L = \phi$?

Important operations on languages -

1) Union: The union of 2 languages L and M denoted by $L \cup M$ is set of strings that are either in L or in M or both

eg. If $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$

$L \cup M = \{\epsilon, 001, 10, 111\}$

29 July 2025

Quick Work

Page No.:

Date:

M T W T F S S

Automata Theory

2) # Concatenation:

Concatenation of languages L and M denoted by $L.M$ or just LM is the set of strings that can be formed by taking any string in L and concatenating it with any string in M .

eg. If $L = \{001, 10, 111\}$ and $M = \{\epsilon, 001\}$
then $LM = \{001, 10, 111, 001001, 10001, 111001\}$

$\epsilon = \text{NULL}$

↑
0
string
length

$$A = \{0, 1\} \quad B = \{a, b\}$$

$$A \circ B = \{0a, 0b, 1a, 1b\}$$

$$L = \{001\epsilon,$$

$$|\epsilon| = 0$$

$$|001\epsilon\epsilon| = 3$$

3) Kleene Closure:

Closure of a language L is denoted by L^* and represents the set of those strings that can be formed by taking any number of strings from L possibly with iterations.

eg. $\{0, 1\}^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

30 JULY 2025

Quick Work

Page No.:

Date:

M T W T F S S

Automata Theory

Phase Structure Grammar -

- 4 tuple $G = (V, T, P, S)$ where

- (i) V is a finite set of strings called variables or non terminals
- (ii) T is a finite set of strings disjoint from V is called terminals

(iii) P is a finite set of rules (or productions) of the form $\text{String 1} \rightarrow \text{String 2}$, where String 1 may be a string of any number of terminals or non terminals but having ~~an~~ at least one ~~non~~ non terminal and String 2 may be a string of all terminals at which point derivation must stop

(iv) S is the element of V , S is the start symbol

eg:

The production $aabbX \rightarrow ab$ says that string ab is a substitution string of string $aabbX$. If we find any patterns of $aabbX$ then we can replace it with ab

Automata Theory -

- Automata Theory studies abstract machines (automata) & problems they can solve
- It is a branch of theoretical computer science & mathematics
- Helps design and analyze computational systems & formal languages

What is an Automaton?

- Mathematical model of a machine with discrete inputs & outputs.
- Moves between different states based on rules
- It processes a string of inputs & checks if it belongs to a specific language

Importance of Automata Theory -

- Forms basis of compiler design, lexical analysis and parsers
- Helps in designing hardware circuits and protocol verification
- Provides insight into the limitations of computation and helps classify problems by complexity

Types of Automata -

- 1) Finite Automata (FA)
 - Deterministic (DFA) and Non-deterministic (NFA) (NFA)
 - Recognizes Regular languages
 - Simplest type of automaton with finite memory
- 2) Pushdown Automata (PDA)
 - Uses a stack as memory
 - Recognizes Context Free languages
- 3) Linear Bounded Automata (LBA)
 - A type of Turing machine with limited tape
 - Recognizes Context Sensitive languages
- 4) Turing Machines (TM)
 - Most powerful automaton model
 - Recognizes Recursively Enumerable languages
 - Models general purpose computation

Application of Automata Theory -

- 1) Formal verification of software and hardware systems
- 2) Text processing and pattern matching
- 3) Natural language processing
- 4) AI
- 5) Cryptography

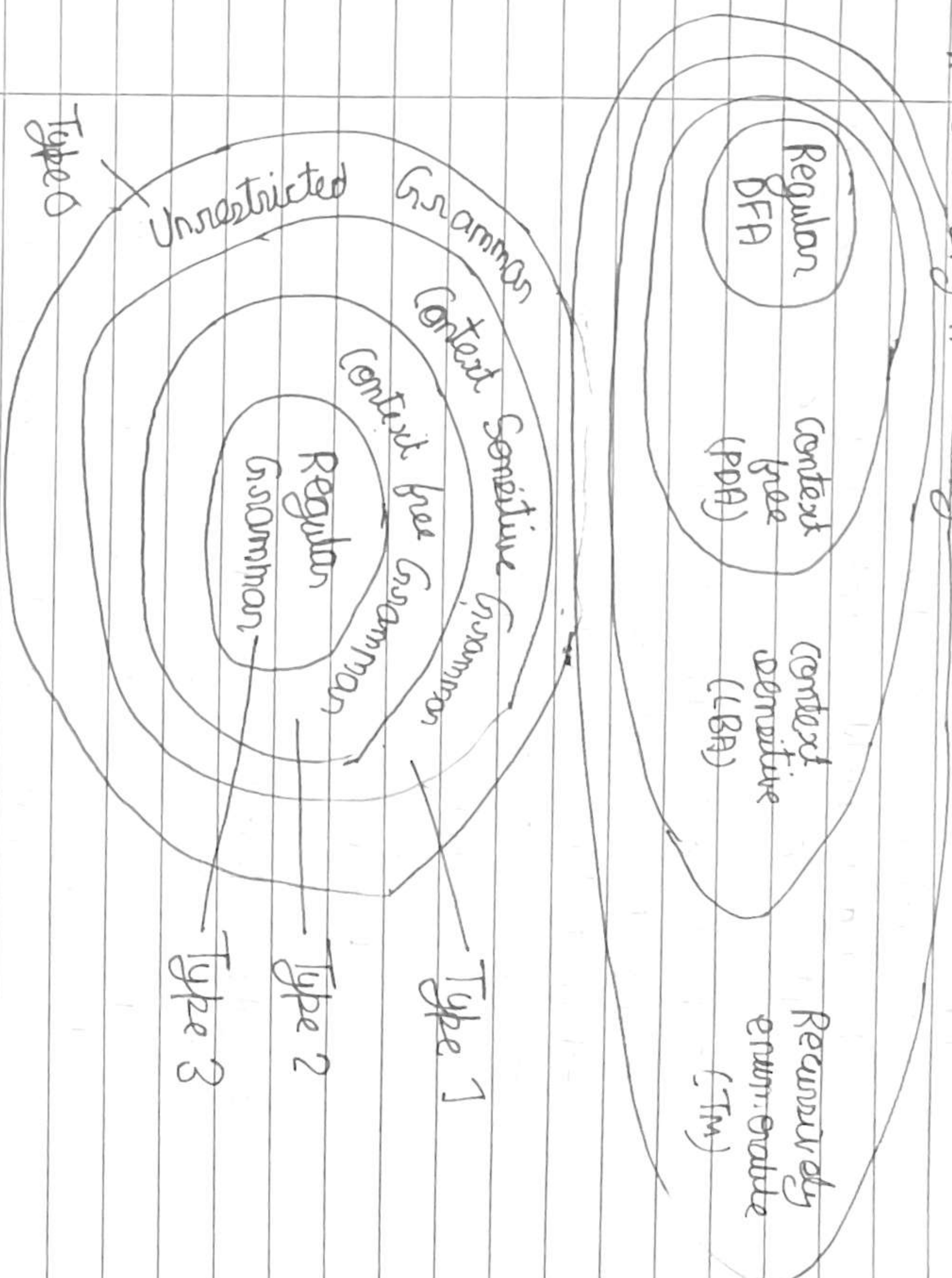
Relation to Formal Languages -

Each type of automaton is associated with a class of formal languages:

- FA $\rightarrow$ Regular languages
- PDA $\rightarrow$ Context Free languages
- LBA $\rightarrow$ Context Sensitive languages
- TM $\rightarrow$ Recursively Enumerable languages

These relationships are organized in the Chomsky hierarchy.

Chomsky Hierarchy -



Chomsky Unrestricted (Type 0) Grammar -

- No restrictions on rules
- Productions: $\alpha \rightarrow \beta$ ($\alpha \neq \epsilon$)
- Most general and powerful
- Can simulate any computation (Turing Machine)
- eg $bAa \rightarrow aa$
- $S \rightarrow s$

Type-1 Context Sensitive Grammar -

- Rule format: $x\alpha y \rightarrow x\beta y$ (length of output $\geq$ input)
- α cannot be shorter than β
- Erasing a variable is not allowed
- Captures context in both directions $\nearrow$ eg $S \rightarrow AT$
 $T \rightarrow xy$
 $A \rightarrow a$

Type-2 Context Free Grammar -

- Rule: $A \rightarrow \alpha$ (only one variable on LHS)
- Doesn't depend on surrounding symbols
- Used in programming languages
- eg $A \rightarrow aBb$
 $A \rightarrow b$
 $B \rightarrow a$

Type-3 Regular Grammar -

- Most restricted grammar type
- Rule: $A \rightarrow a$ or $A \rightarrow aB$ (only one variable at RHS end)
- Used in finite automata & lexical analysis
- eg $A \rightarrow xy$

Operations On Formal Language -

- Used to construct new languages from existing ones
- Analogous to set operations, but applied to sets of strings (languages)

- Union ($L_1 \cup L_2$)
- Concatenation ($L_1 L_2$ or $L_1 L_2$)
- Kleene Star (L^*)
- Kleene Plus (L^+)
- Intersection ($L_1 \cap L_2$)
- Difference ($L_1 - L_2$)
- Complement ($\bar{L}$ or L^c)

1) Union ($L_1 \cup L_2$)

All strings that are in either L_1 , L_2 or both

eg- $L_1 = \{a, ab\}$ $L_2 = \{a, ab, b, ba\}$
 $L_1 \cup L_2 = \{a, ab, b, ba\}$

2) Concatenation ($L_1 L_2$ or $L_1 L_2$)

All strings formed by concatenating a string from L_1 with a string from L_2

eg- $L_1 = \{a, b\}$ $L_2 = \{a, ab\}$
 $L_1 L_2 = \{aa, ab, ba, bab\}$

3) Kleene Star (L^*)

Set of strings formed by concatenating zero or more strings from L

eg- $L = \{a\}$
 $L^* = \{\epsilon, a, aa, aaa, \dots\}$ (OR)
 $L = \{ab\}$
 $L^* = \{\epsilon, ab, abab, ababab, \dots\}$

Chom Unrestricted (Type 0) Grammar -

- No restrictions on rules
- Productions: $\alpha \rightarrow \beta$ ($\alpha \neq \text{empty}$)
- Most general and powerful
- Can simulate any computation (Turing Machine)
- eg $bAa \rightarrow aa$
- $S \rightarrow s$

Type-1 Context Sensitive Grammar -

- Rule format: $xAy \rightarrow x\alpha y$ (length of output $\geq$ input)
- α cannot be shorter than A
- Ensuring a variable is not allowed
- Captures context in both directions $\nearrow$ eg $S \rightarrow AT$
 $T \rightarrow xy$
 $A \rightarrow a$

Type-2 Context Free Grammar -

- Rule: $A \rightarrow \alpha$ (only one variable on LHS)
- Doesn't depend on surrounding symbols
- Used in programming languages
- eg $A \rightarrow aBb$
- $A \rightarrow b$
- $B \rightarrow a$

Type-3 Regular Grammar -

- Most restricted grammar type
- Rule: $A \rightarrow a$ or $A \rightarrow aB$ (only one variable at RHS end)
- Used in finite automata & lexical analysis
- eg $A \rightarrow xy$

Operations On Formal Language -

- Used to construct new languages from existing ones
- Analogous to set operations, but applied to sets of strings (languages)

- (i) Union ($L_1 \cup L_2$)
- (ii) Concatenation ($L_1 L_2$ or $L_1 L_2$)
- (iii) Kleene Star (L^*)
- (iv) Kleene Plus (L^+)
- (v) Intersection ($L_1 \cap L_2$)
- (vi) Difference ($L_1 - L_2$)
- (vii) Complement ($\neg L_1$ or L_1^c)

1) Union ($L_1 \cup L_2$)

All strings that are in either in L_1, L_2 or both

eg- $L_1 = \{a, ab\}$ $L_2 = \{a, ab, b, ba\}$

$L_2 = \{b, ba\}$

2) Concatenation ($L_1 L_2$ or $L_1 L_2$)

All strings formed by concatenating a string from L_1 with a string from L_2

eg- $L_1 = \{a, b\}$ $L_2 = \{a, ab, ba, bab\}$

$L_1 L_2 = \{aa, aab, ba, bab\}$

3) Kleene Star (L^*)

Set of strings formed by concatenating zero or more strings from L

eg- $L = \{a\}$

$L^* = \{\epsilon, a, aa, aaa, \dots\}$

(OR)

$L = \{ab\}$

$L^* = \{\epsilon, ab, abab, ababab, \dots\}$

4) Kleene Plus (L^+)

Like Kleene Star, but excludes ϵ (empty string)
So, it's one or more concatenations

eg- $L = \{a\}$
 $L^+ = \{a, aa, aaa, \dots\}$

5) Intersection ($L_1 \cap L_2$)

All strings that are common to both L_1 and L_2

eg- $L_1 = \{a, ab, abc\}$
 $L_2 = \{ab, abc, b\}$
 $L_1 \cap L_2 = \{ab, abc\}$

6) Difference ($L_1 - L_2$)

All strings that are in L_1 but not in L_2

eg- $L_1 = \{a, b, ab, ba\}$
 $L_2 = \{ab, ba\}$
 $L_1 - L_2 = \{a, b\}$

7) Complement ($\neg L$ or L^c)

All strings over the alphabet Σ that are not in L

eg- $\Sigma = \{a, b\}$
 $L = \{a, b, ab\}$
 $L^c =$ all strings over $\{a, b\}$ except $\{a, b, ab\}$
like $\{ \epsilon, aa, bb, ba, aab, \dots \}$

Applications of Formal Languages -

1) Regular Languages (Type 3)

- Recognised by: Finite Automata
- (i) Lexical analysis
- (ii) Text searching
- (iii) Input validation
- (iv) Syntax highlighting
- (v) Network protocols

2) Context Free Languages (Type 2)

- Recognised by: Pushdown Automata
- (i) Programming Language Syntax
- (ii) Parsing tools
- (iii) Expression evaluation
- (iv) Natural Language Processing (simplified)

3) Context Sensitive Languages

- Recognised by: Linear Bounded Automata
- (i) Natural language processing
- (ii) Compiler design (semantic analysis)
- (iii) Formal verification

4) Recursively Enumerable Languages (Type 0)

- Recognised by: Turing machines
- (i) General Problem Solving
- (ii) AI Planning
- (iii) Automated Theorem Proving
- (iv) Simulation of computation

9 AUG 2025

Quick Work

Page No.
Date
M T W T F S S

Finite Automaton -

- Formal System
- Remembers only a finite amount of information
- Info related to represented by its states
- State changes in response to inputs
- Rules that tell how the state changes in response to inputs are called transitions

Finite automata -

- Finite automata recognise patterns
- Input string of symbols $\rightarrow$ state changes occur on matching symbols
- On transition: move to next state or stay in same state
- States: Accept or Reject, accept if final state reached after full input

Features -

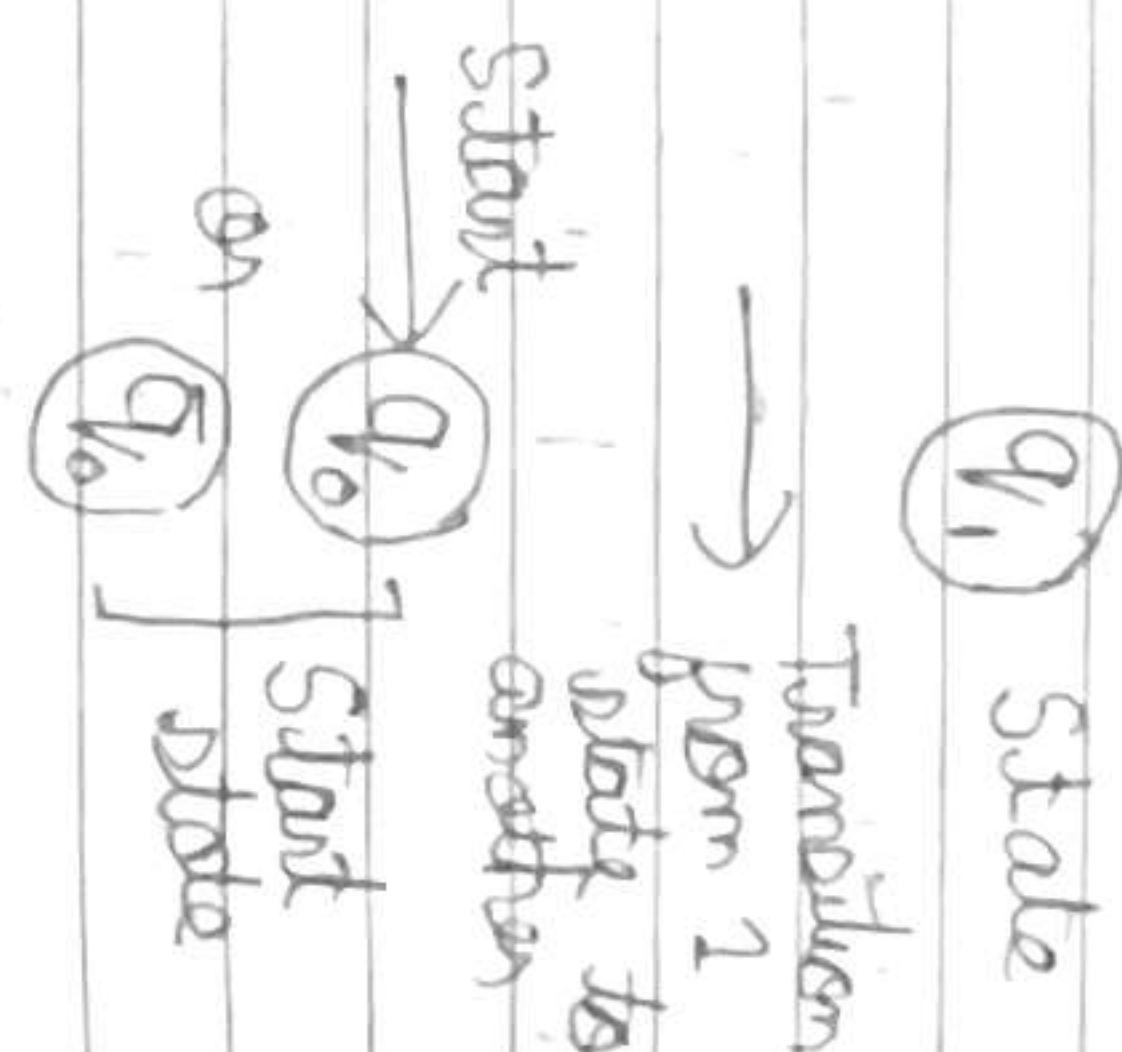
- Input: Set of symbols or characters provided to the machine
- Output: Accept or reject based on input pattern
- States of automata: Conditions/configurations of the machine
- State Relation: the transitions between states
- Output Relation: Based on the final state, output decision is made

Types of Finite Automata -

- (i) DFA (deterministic finite automata)
- (ii) NFA (non deterministic FA)

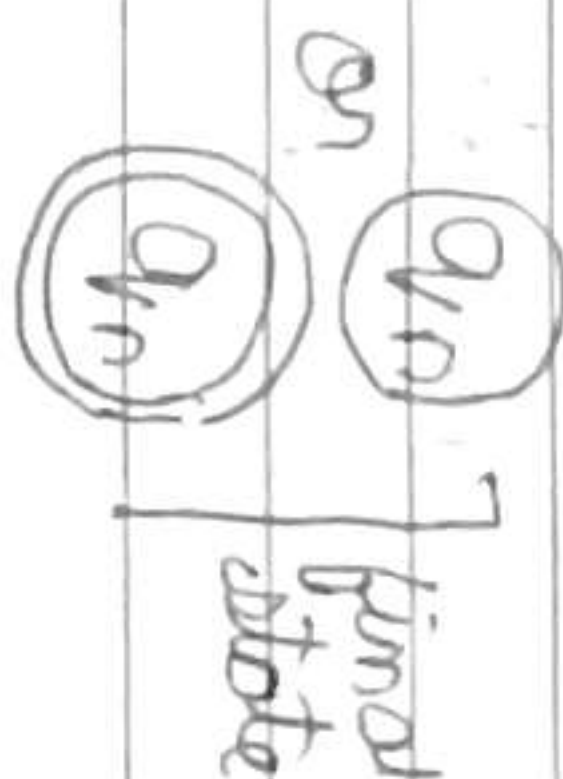
Transition Diagram -

- Node for each state in Q (represented by a circle)
- Directed edge from node q_i to node p labeled a if $\delta(q_i, a) = p$
- In the start state, there is an arrow with no source
- Accepting states or final states are indicated by a double circle.



Transition Function -

- Takes 2 arguments: a state and an input symbol
- $\delta(q, a)$ = state that the DFA goes to when it is in state q and input a is received
- NOTE: If no valid transition exists, add a dead state



1) Deterministic Finite Automata -

- Deterministic refers to the uniqueness of the computation. In the DFA, machine goes to one state only for a particular input character
- DFA does not accept null: move/transitions
- DFA consists of 5 tuples: $\{Q, \Sigma, q, F, \delta\}$
- Q = set of all states
- Σ = set of input symbols
- q = initial state
- F = set of final state
- δ = Transition function, defined as $\delta: Q \times \Sigma \rightarrow Q$

Quick Work

Page No.
Date
M T W T F S S



15 AUG 2025

Finite Automata

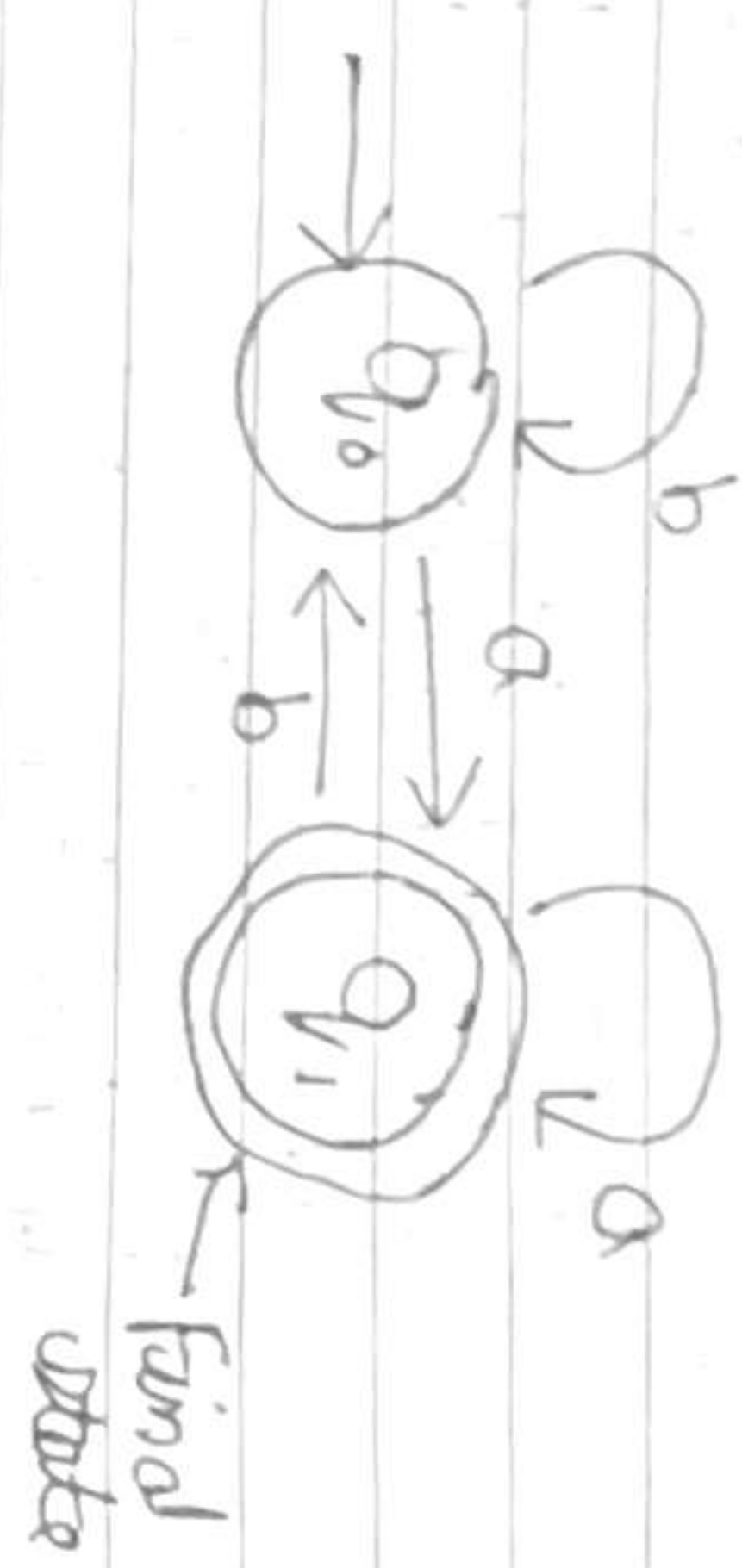
eg. Construct a DFA that accepts all strings ending with 'a'

Given:

Input $\rightarrow \Sigma = \{a, b\}$

States $\rightarrow Q = \{q_0, q_1\}$

Final State $\rightarrow F = \{q_1\}$



Q. Write a language that accepts all the combination of 'a' and 'b'.

alphabet $\Sigma = \{a, b\}$

Ans

$g^* = \{\epsilon, a, aa, aaa, \dots\}$
 $g^+ = \{a, aa, aaa, \dots\}$

$G \rightarrow$ quadruple

$G = \{V, T, P, S\}$



Variable
Terminal
non terminal a, b, c
A, B, C

is a grammar which consists of set of production rule. It is used to generate the string of a language.

is a grammar which consists of set of production rule. It is used to generate the string of a language.

T is a terminal symbol
It is denoted by lower case

V is a non terminal variable/symbol. It is denoted by upper case

P is a set of production rules which is used for replacing non terminals with terminals

Start Symbol: used to derive a string

$S \rightarrow aSb \rightarrow$ if you start with S you can replace it with a, then again S, then b

$S \rightarrow \epsilon$ allows termination (ending the derivation)

$S \rightarrow aSb$
$S \rightarrow bSa$
$S \rightarrow \epsilon$

$\rightarrow$ full expansion

Strings generated:

Using S

$\epsilon \rightarrow$ we get empty string ""

$aSb \rightarrow ab$

$aSb \rightarrow aaSbb \rightarrow aabb$

$bSa \rightarrow ba$

$bSa \rightarrow bbSaa \rightarrow bbaa$

So this grammar generates palindromes

$A \rightarrow aBb$ Begins with a & ends with b alongside having non terminal B inside

$A \rightarrow aaaa$

as New, expand B in different ways:

Case 1: $B \rightarrow aa$

$A \rightarrow aBb \rightarrow aabbb$

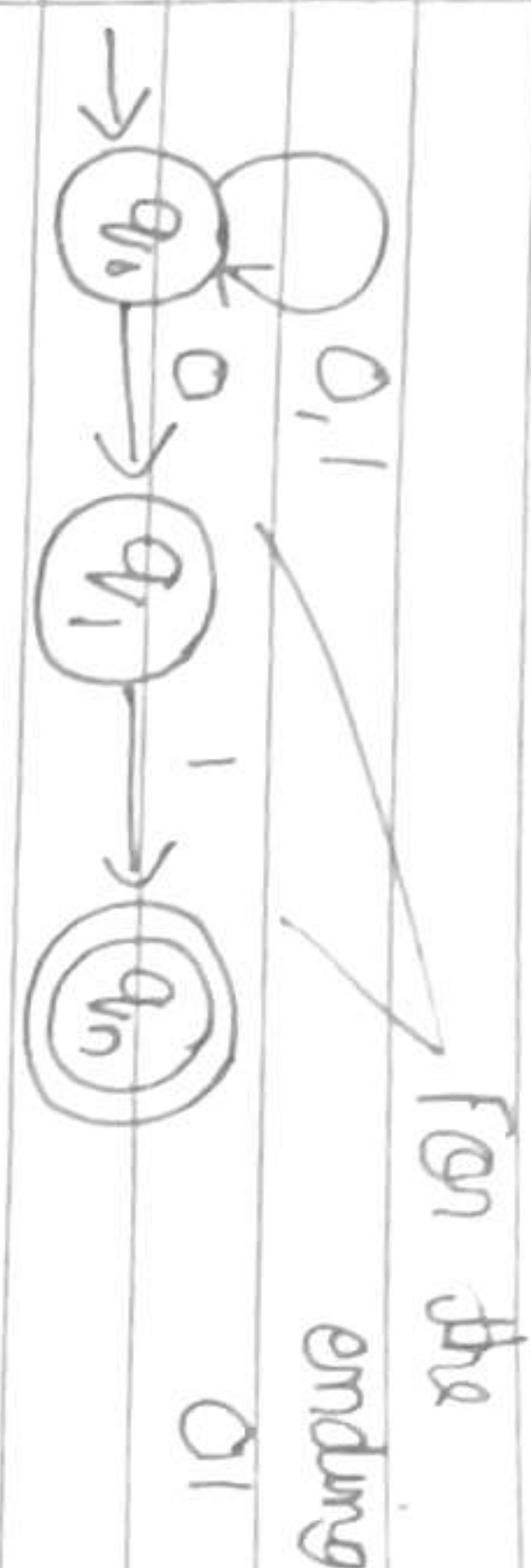
Case 2: $B \rightarrow bb$

$A \rightarrow aBb \rightarrow abbbb$

Case 3: $B \rightarrow \epsilon$

$A \rightarrow aBb \rightarrow abb$

Q Construct a DFA of a string $\{0,1\}$ that ends with 01



self loop

can create any numbers like 0000 01

1010 01
0010 01

DFA

Advantages:

- Deterministic behavior
- Fast execution
- Easy to implement
- Unique new of computation path

Disadvantages:

- State explosion (requires exponentially more states)
- Harder to construct for complex patterns

NFA

Advantages:

- Compact representation
- Easier to construct
- Support for ϵ -transitions

Disadvantages:

- Non deterministic
- Slower execution
- More theoretical

Grammar: A grammar 'G' is defined as quadruple:

$G = \{V, T, P, S\}$

Variable $\rightarrow$ Production
Terminal $\rightarrow$ Rule

$S \rightarrow aSb / \epsilon$

$\epsilon, aSb, aaSb, aaaSbbb$

ϵ, aSb (after inserting instead of S), a^2Sb^2, a^3Sb^3
 $a^2b^2, a^3b^3, \dots, a^n b^n$

$n \geq 0$

Q. Can we generate abab using $S \rightarrow aSb / \epsilon$

aSb aSb
 $a aSb b$ $a b$
 $a a b b$

No, we cannot generate abab

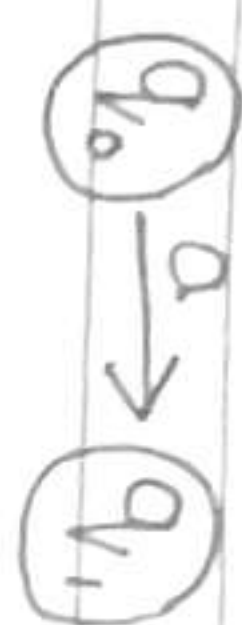
Q.

$S \rightarrow SS$ $\epsilon, aSb bSa$
 $S \rightarrow aSb$ $abba$
 $S \rightarrow bSa$
 $S \rightarrow \epsilon$ after putting ϵ

Language $\Rightarrow \boxed{n_a(w) = n_b(w)}$

single state

DFA($\emptyset \leq \delta, q_0, F$)



Q = Set of finite states

Σ = Set of alphabets

δ = Transition

q_0 = Start state

F = ~~Final~~ Set of final states

$F \subseteq Q$

Strings ~~the~~ starting with a {a, aa, aaa, ab, ba

we can't use self loop with b

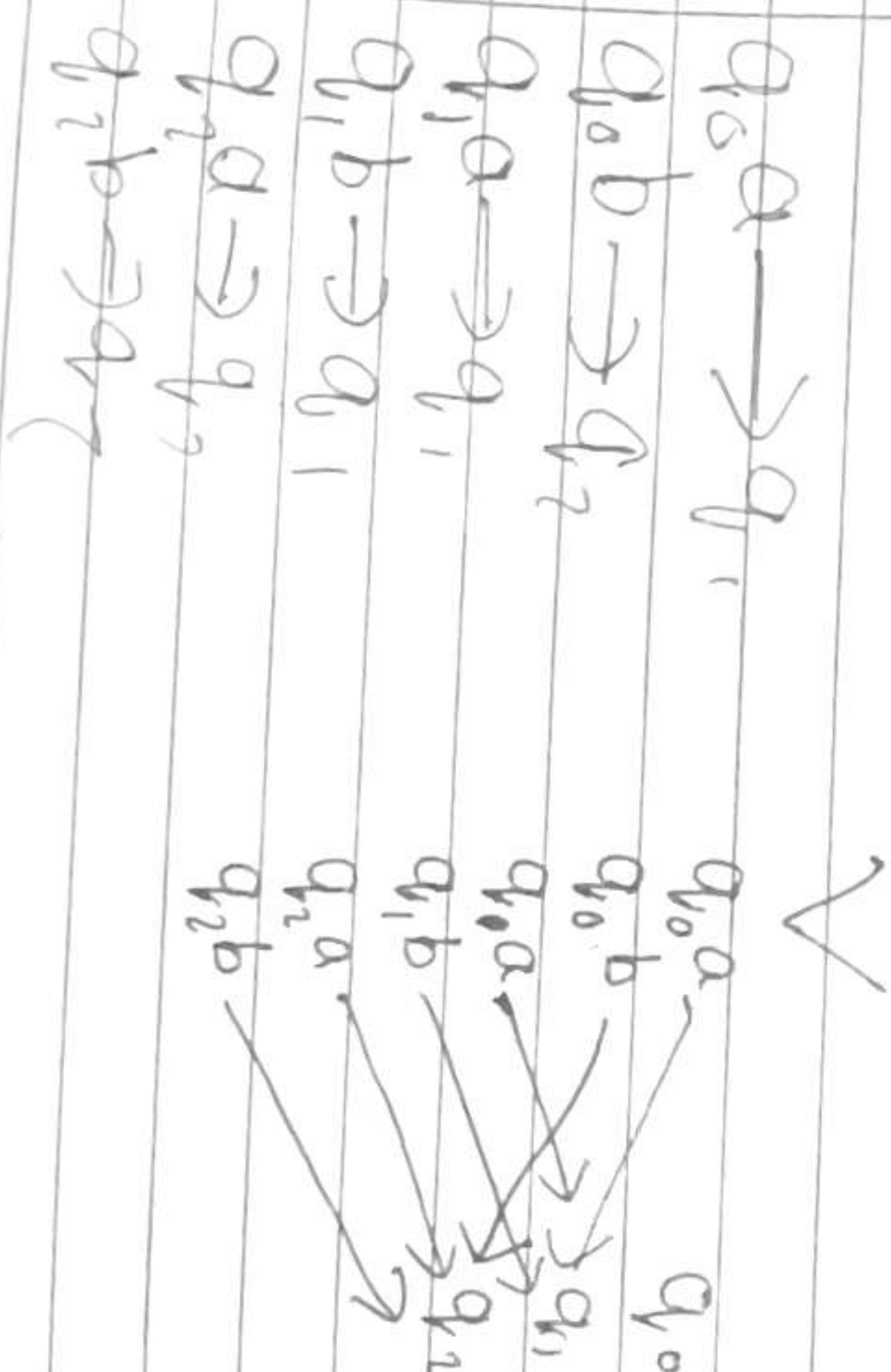


since anything

can come after just a

$S: \emptyset \times \Sigma \rightarrow \emptyset$

q_0 (a,b)
 q_1
 q_2



Q. Construct a DFA which accepts a language of all strings

(a) containing 'a'

(b) ~~containing~~ ending with 'a'

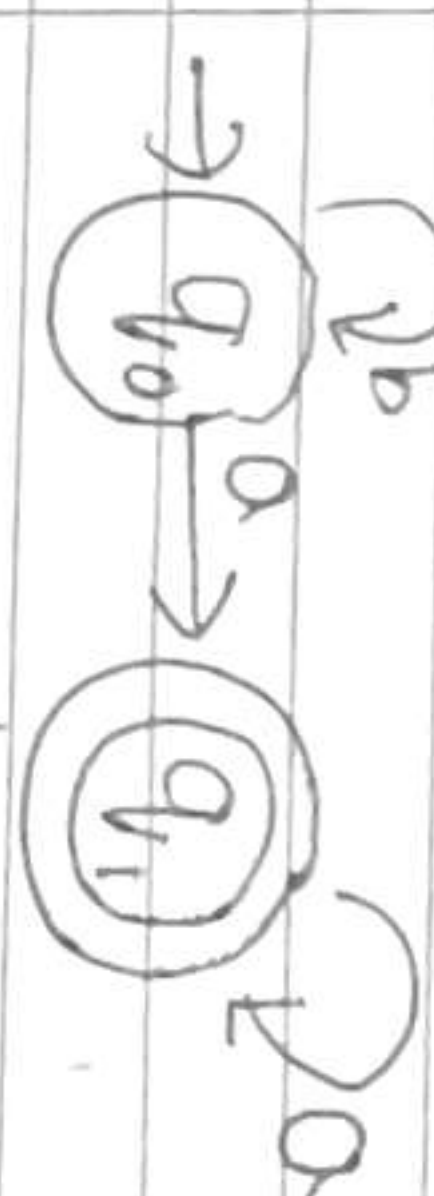
$\Sigma = \{a, b\}$

Ans

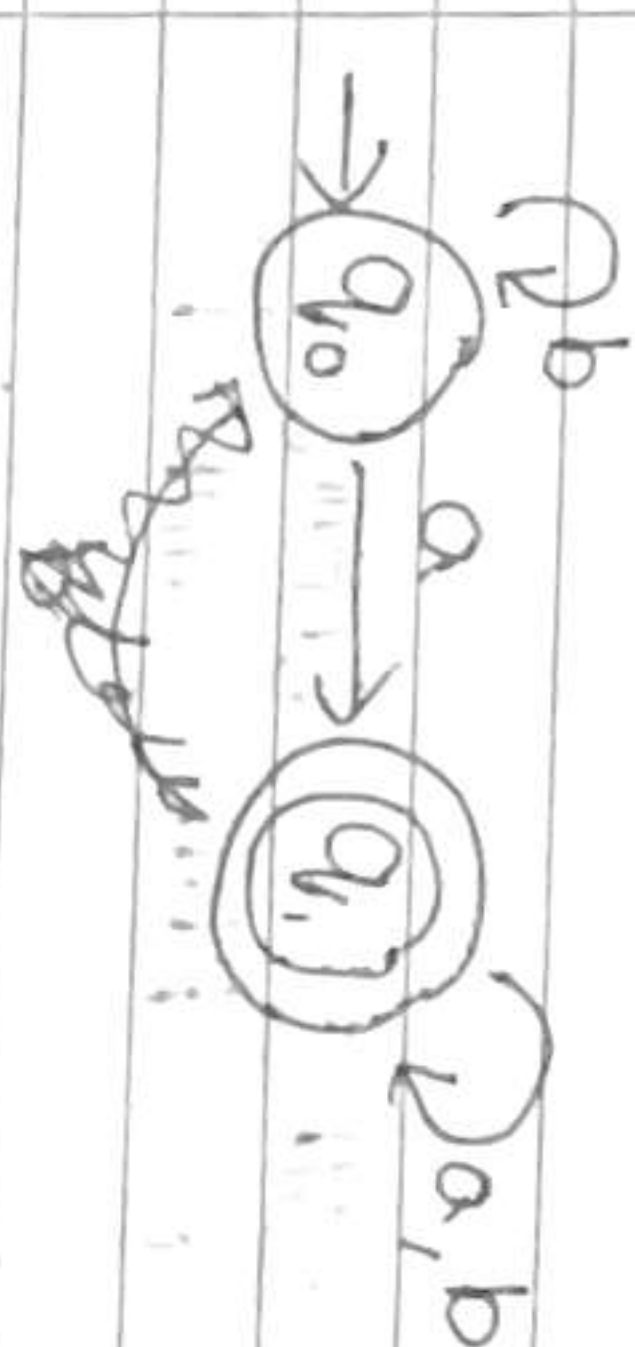
(a)

$L = \{a, aa, ba, aaa, ab, aab\}$

First design the minimum string

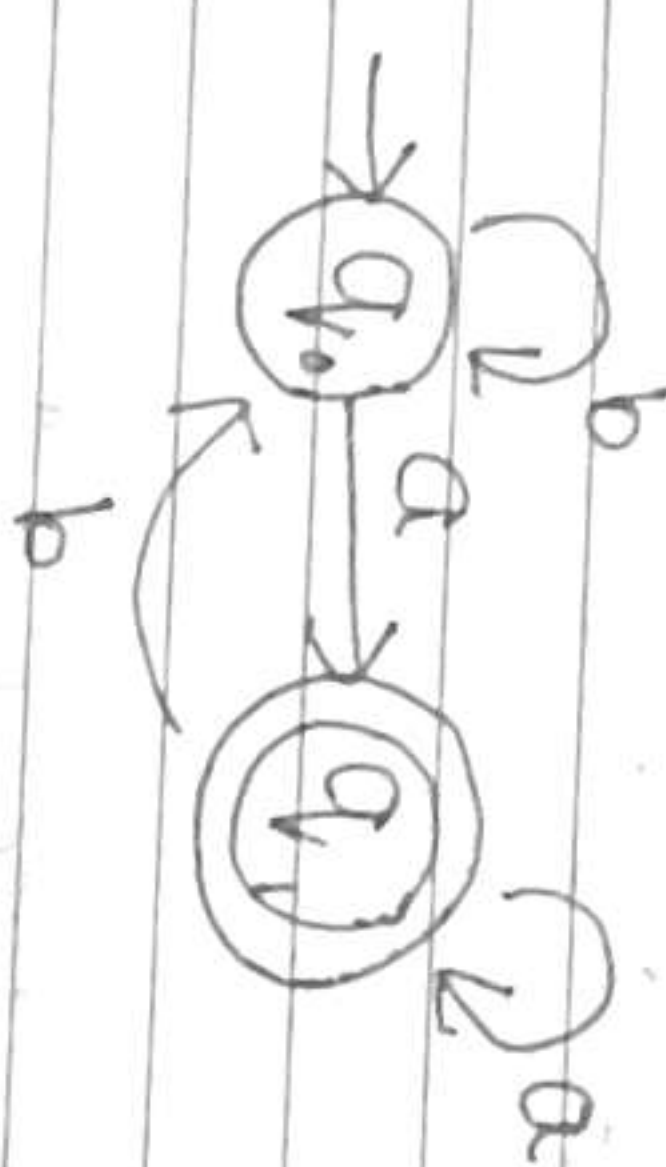


But for aaba



(b)

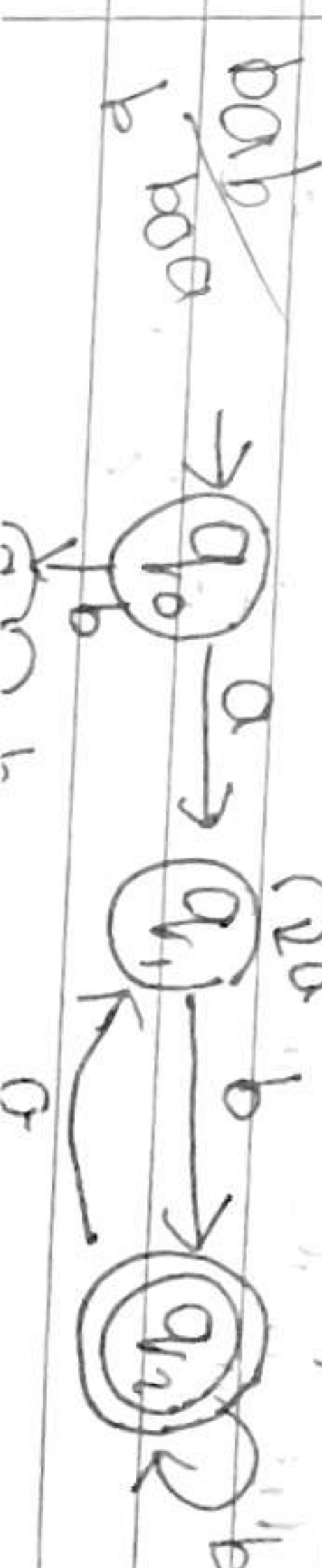
$L = \{a, aa, aaa, ba, baba, aba, abab, abba, ababa, ababb\}$



Q.

Construct a DFA which accepts a language of all strings starting with 'a' and ending with 'b'

$L = \{ab, abab, ababab, aabbb, aabbb\}$



Q Construct a DFA which accepts a language of all strings starting with 'a' or not ending with 'b'.

Ans $L = \{ \lambda, a, b, ba \}$

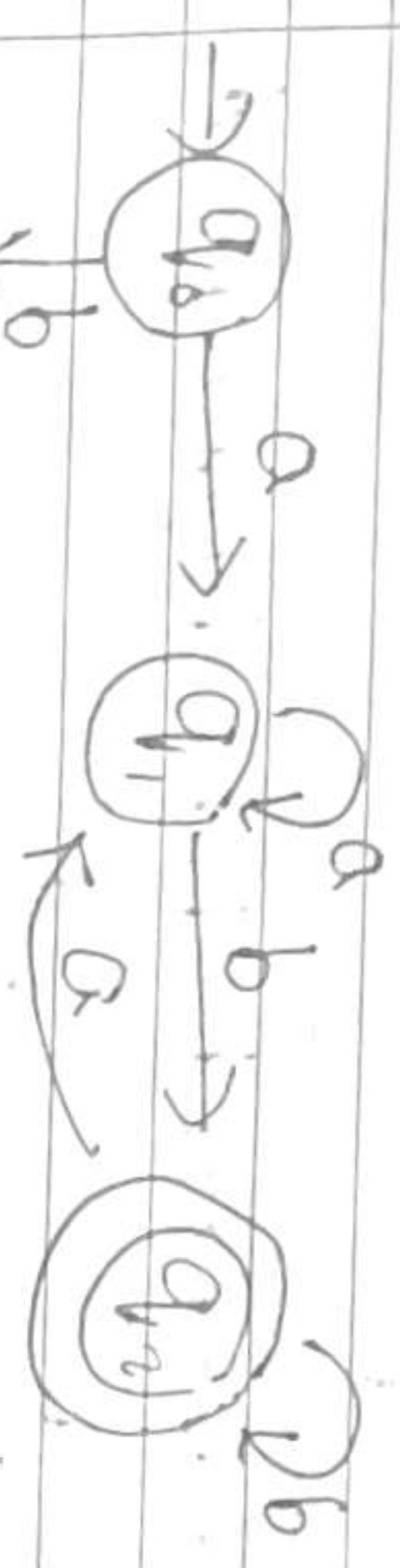
For next questions, use De Morgan's theorem

$$(A \cup B)^c = A^c \cap B^c$$

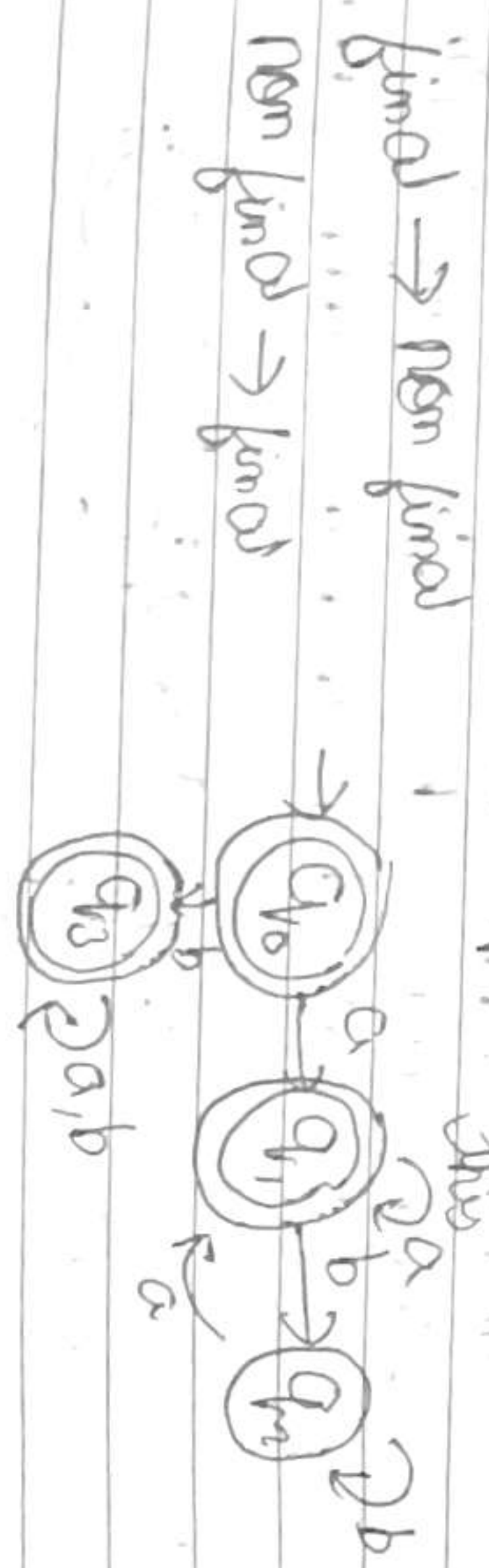
$A = \text{not starting with 'a'}$
 $B = \text{not ending with 'b'}$

$$(A \cup B)^c = A^c \cap B^c$$

Starting with 'a' AND ending with 'b'



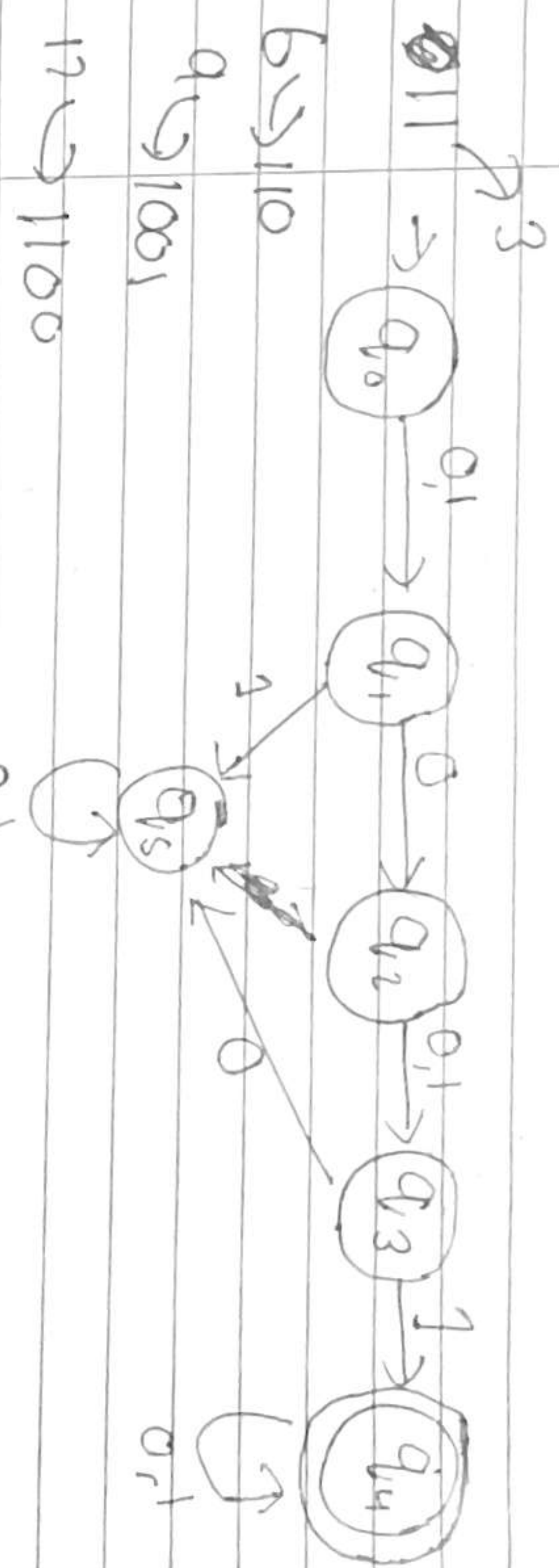
We need to take complement of this



Q Construct a DFA which accepts all strings even (0,1) in which second symbol is '0' and fourth symbol '1'.

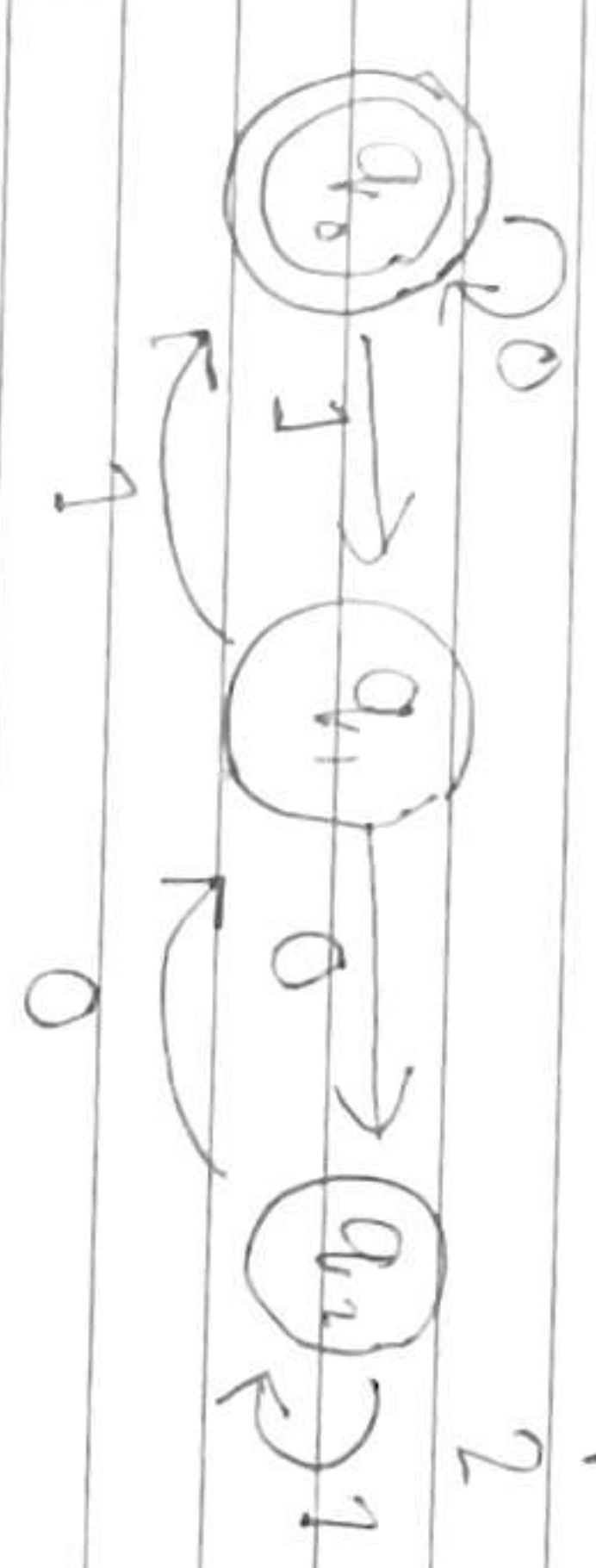
Ans $0-1$ 4 = Min length of string
0/1 0/1

Min states = $n+1$ For DFA: $n+1+1$
 $= 4+1$
 $= 5$ states



Q Construct a DFA which accepts a language of all binary strings divisible by 3 (even $\leq (0,1)$)

Ans If we divide any no. by 3 we get following remainders: 0



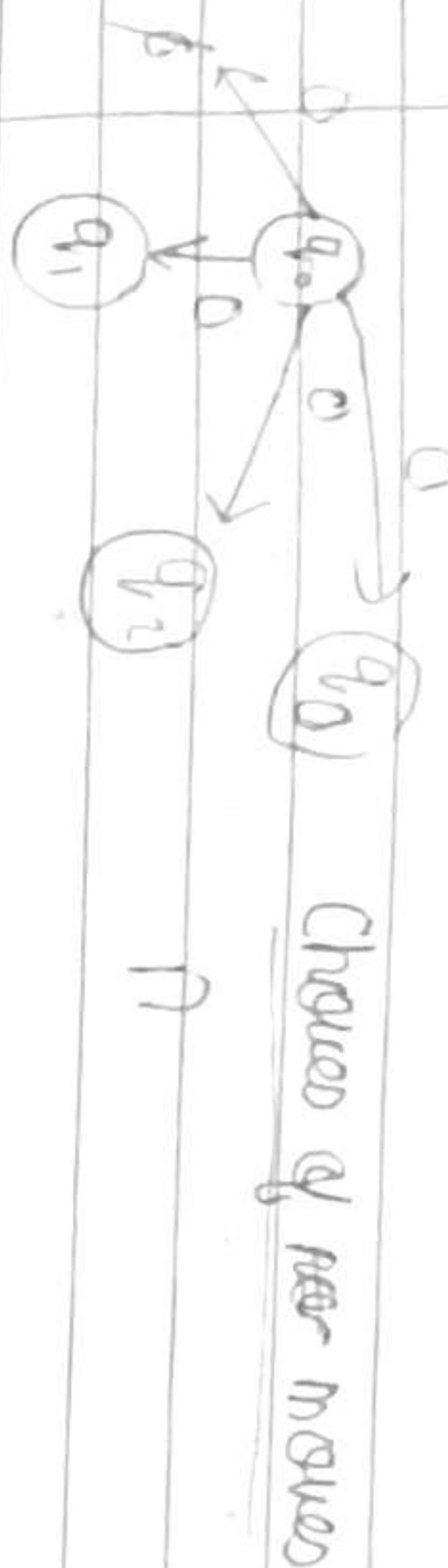
States = remainders

$$100 \frac{4}{3} = 1$$

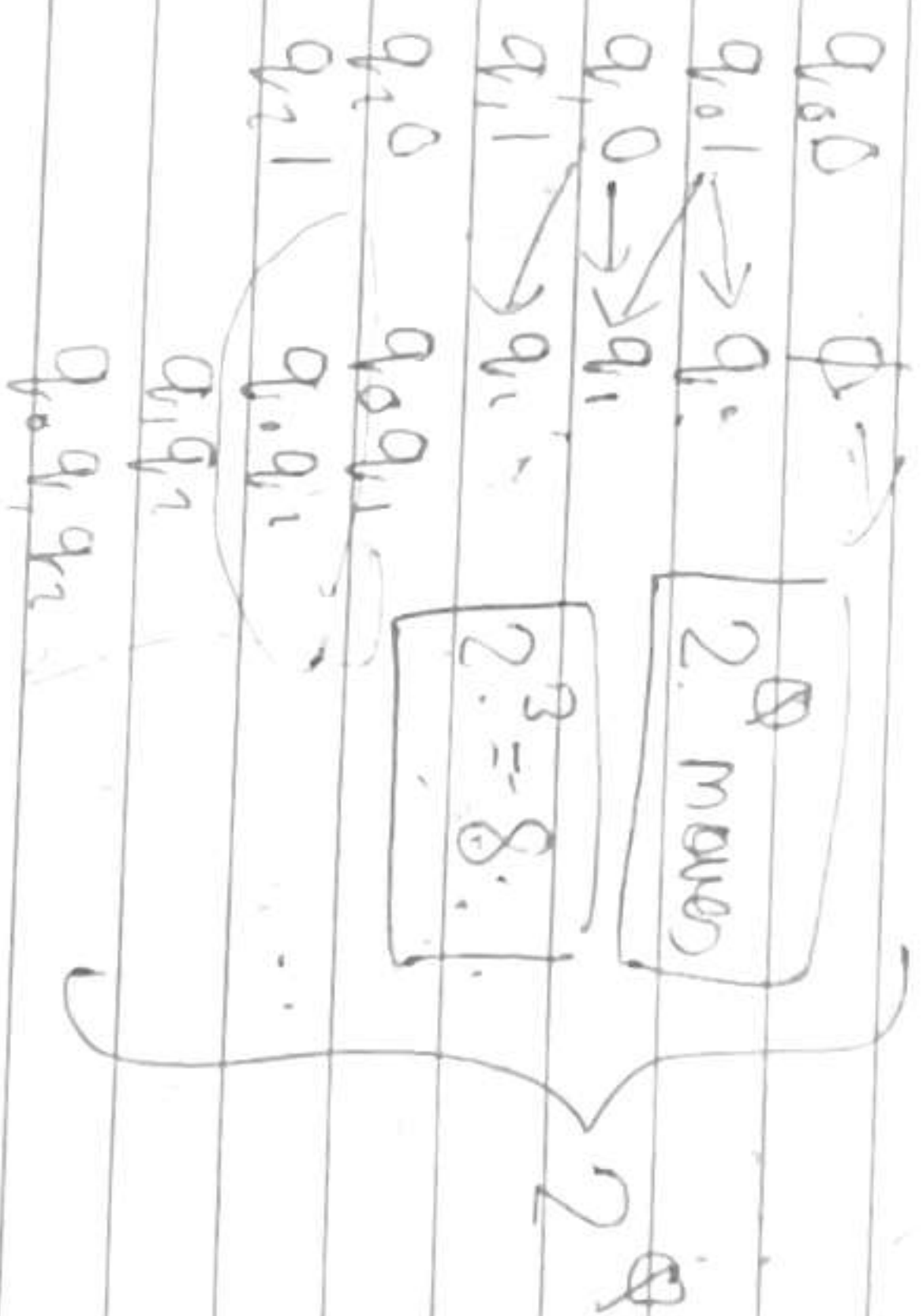
Regular language

NFA/NFA^{*} ($Q, \leq q_0, F, \delta$)

Q - set of finite series
 $F \subseteq Q$



$\delta(Q) \leq 1 = Q$
(q_0, q_1, q_2) (0,1)



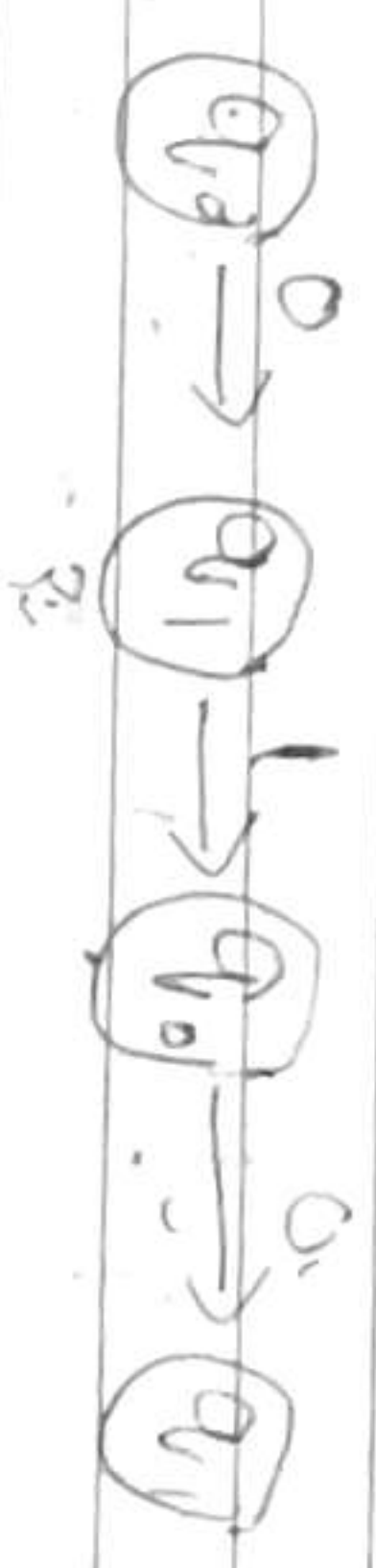
δ is different

DFA vs NFA/NFA^{*}

DFA

NFA

- | | |
|---|---|
| 1) Dead configuration is not allowed | 1) Dead configuration is allowed |
| 2) Multiple choices are not allowed | 2) Multiple choices are available corresponding to an input |
| 3) ε -move is not allowed | 3) ε -move is allowed |
| 4) Digital computers are deterministic | 4) Non deterministic features are not associated with digital computers |
| 5) Designing & understanding is difficult | 5) Designing & understanding is easy |



Dead configuration

NFA $\rightarrow$ DFA

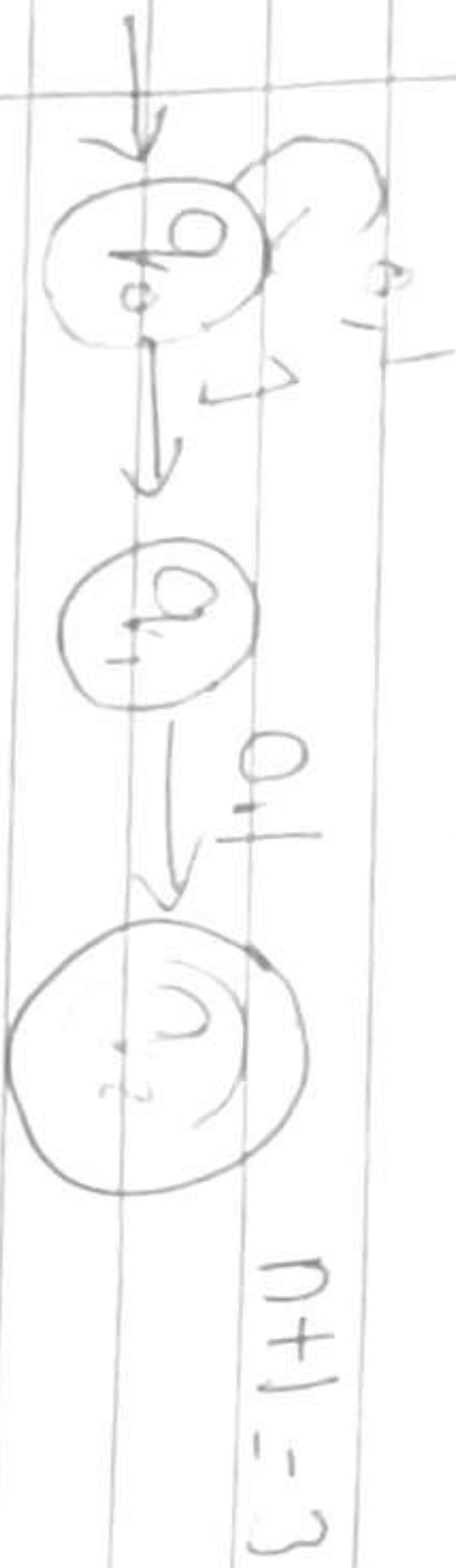
Max states = 2^n

n = states in NFA

NFA of all binary strings in which at 2nd level bit is 1

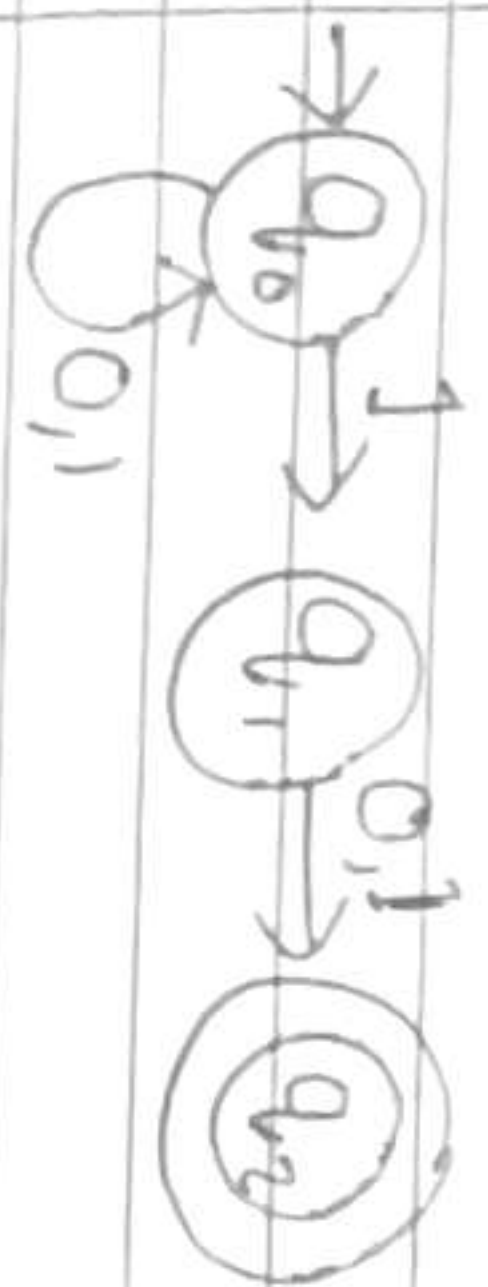
$L = \{ _ _ 1 _ \}$

$(0+1)^* 1 (0+1)^*$



** # NFA to DFA

Q If NFA of all binary strings in which 2nd level bit is 1



Transition table

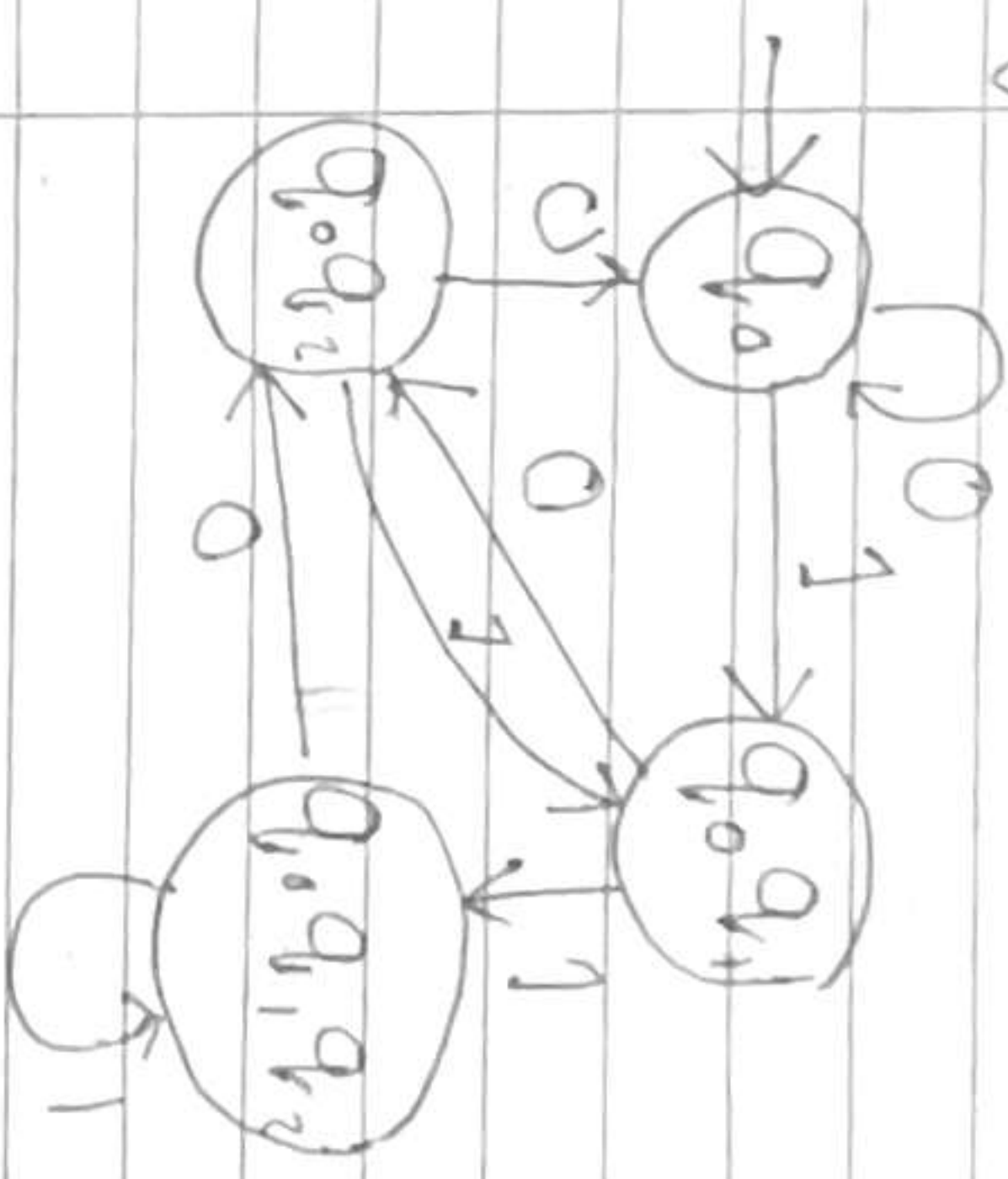
	0	1
$\rightarrow q_0$	q_0	q_1
q_1	q_1	q_2
q_2	q_2	q_2

Please explain this topic + numericals whenever you read this, Thankyou in advance!

copy first state as it is

	0	1
$\rightarrow q_0$	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_1$	$q_0 q_1 q_2$
$q_0 q_1 q_2$	$q_0 q_1 q_2$	$q_0 q_1 q_2$

no coming

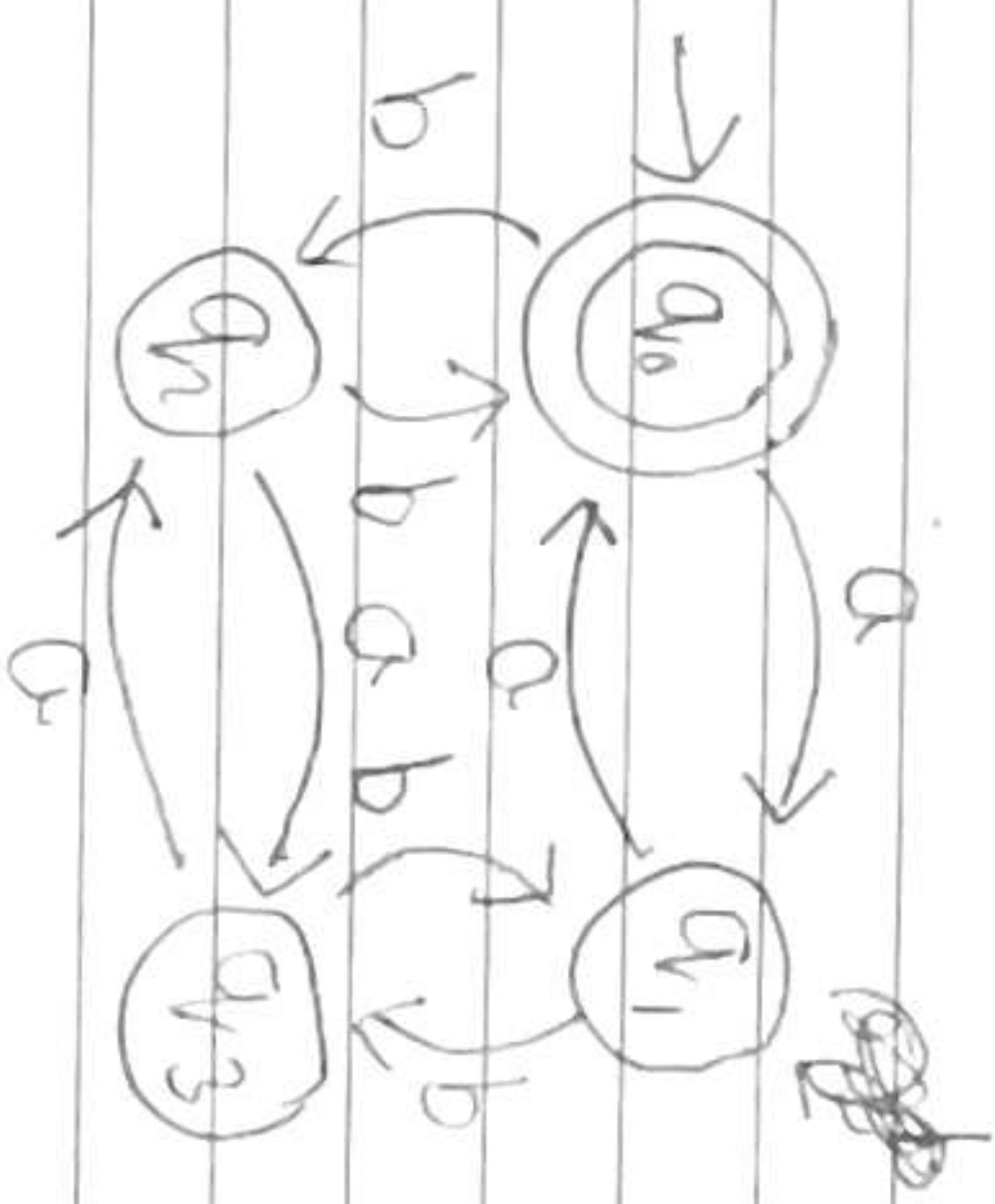


No. of states in DFA > NFA

NFA $\rightarrow n$
DFA $\rightarrow 2^n$ max

Q let $L = \{w | w \text{ has even no. of a's and even no. of b's}\}$

$L = \{ \epsilon, aa, bb, abab, aabb, bbaa, baba, abba, baab \}$



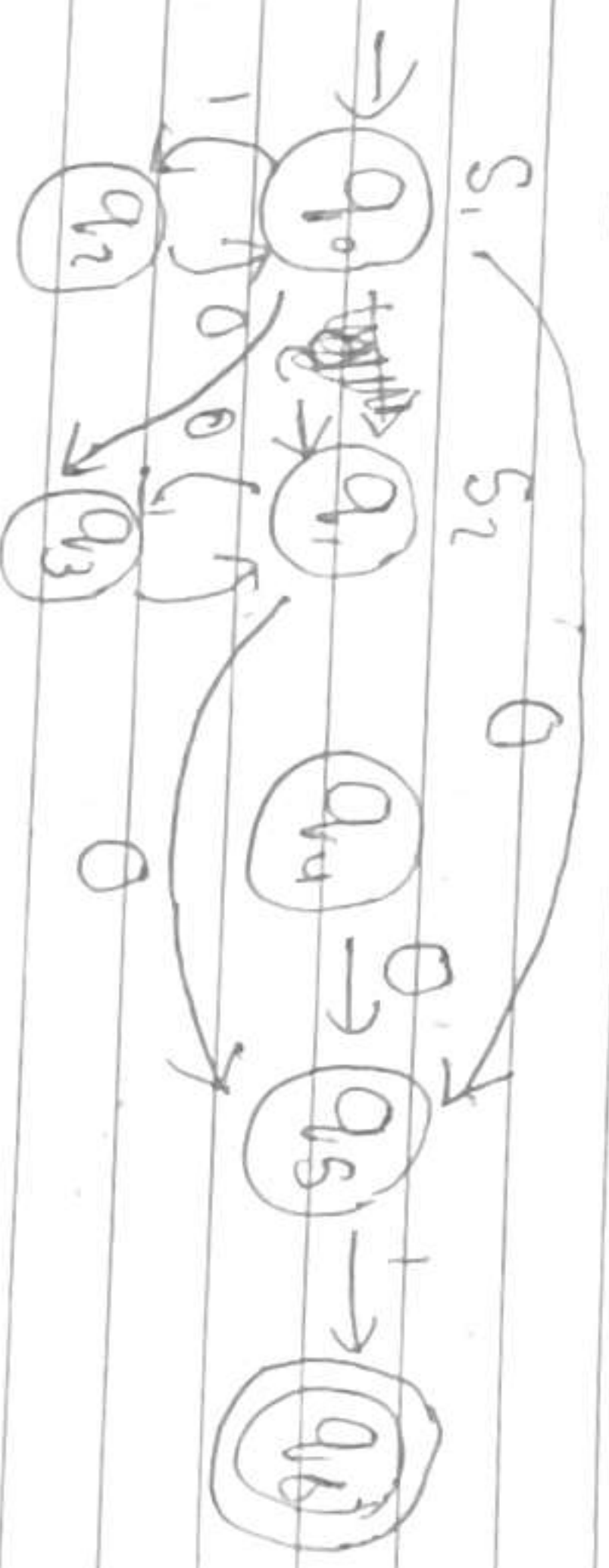
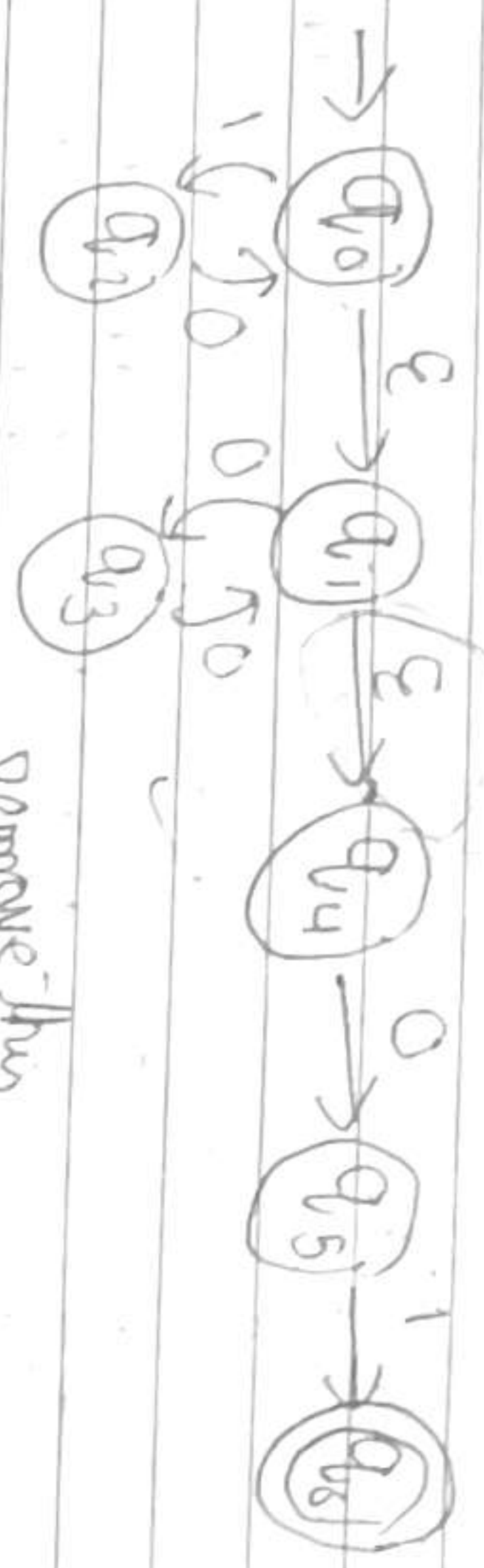
If we make q_1 as final state then add a & even b will be accepted

If we make q_2 as final state, then even a & odd b will be accepted

ϵ -NFA (Episken NFA) $\rightarrow$ Eliminating ϵ moves



- 1) Find all edges starting from S_1
- 2) Duplicate all edges to S_1 without changing edge labels
- 3) If S_1 is initial state, make S_2 initial
- 4) If S_2 is final state, make S_1 final



5) Remove dead state (remove q_4)

$a^1 b^1$
 $a^2 b^1$
 $a^n b^n$
 $a^m b^n$

Limitations of FSA

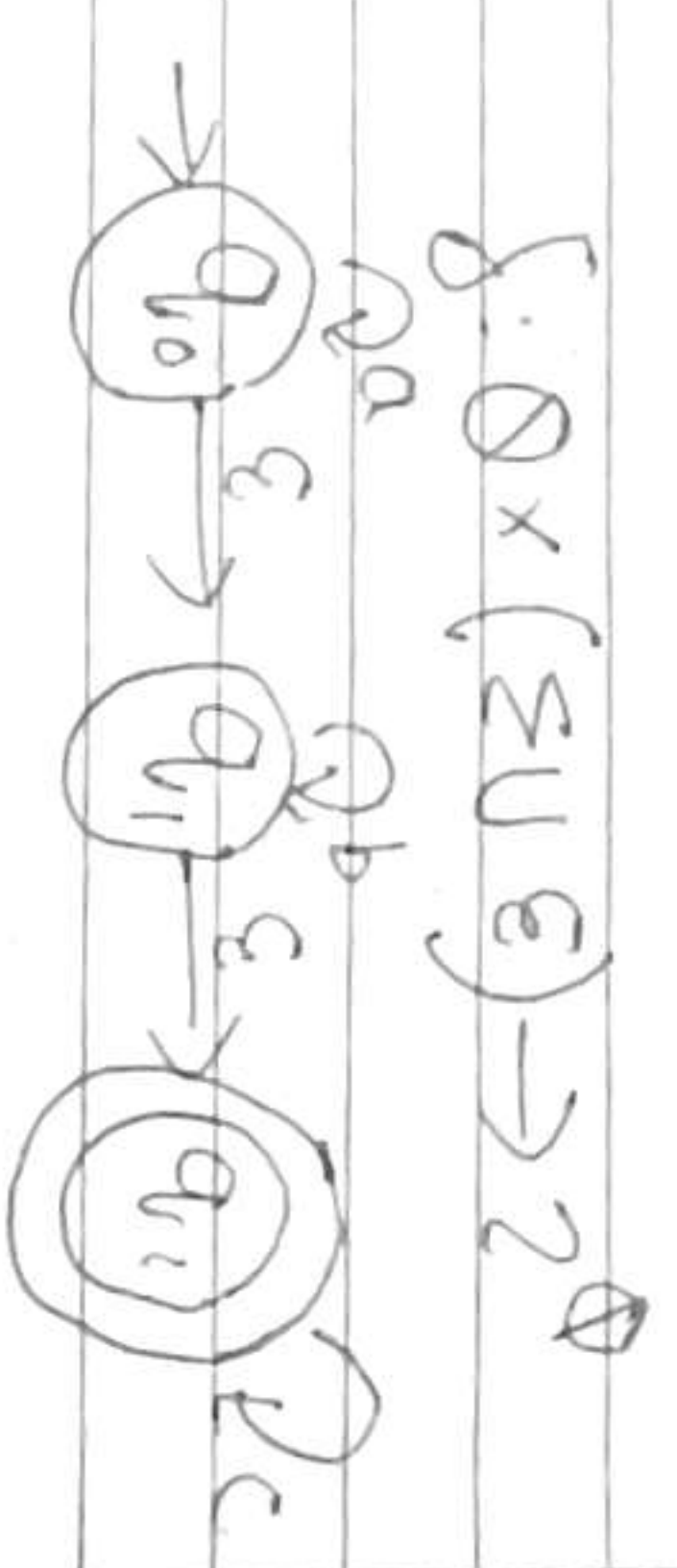
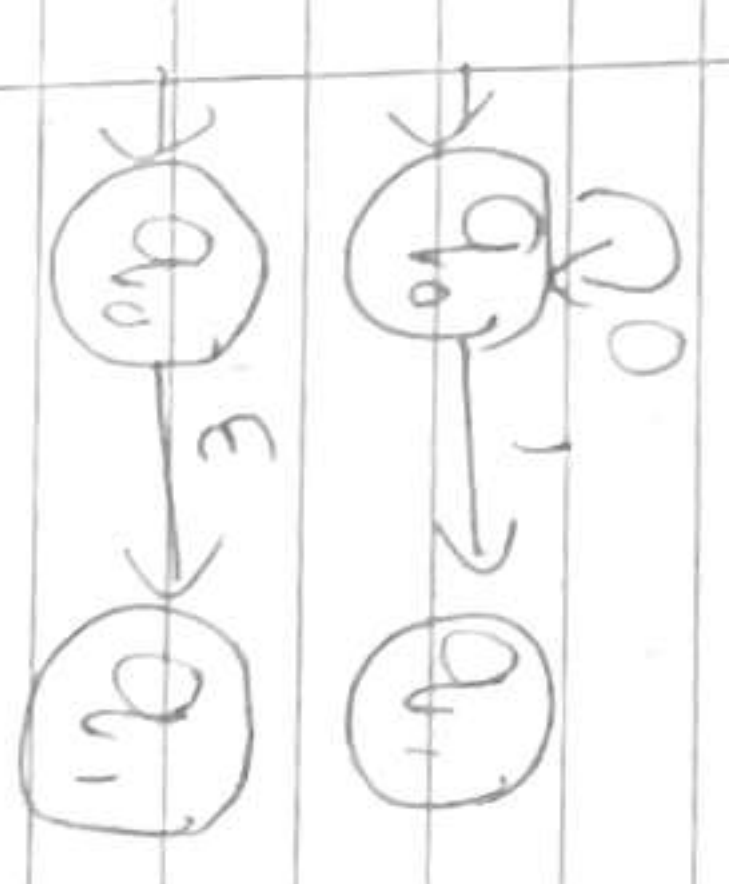
- Limited memory
- Strings without comparison
- Linear Power

Applications of FSA

- Hybrid processor program
- Digital logic design
- Lexical analysis
- Switching circuits
- Text editors
- Game design

ϵ -NFA

Q = Finite set of states
 Σ = input symbols
 q_0 = initial state
 F = set of final states
 δ = Transition function



8 transitions possible

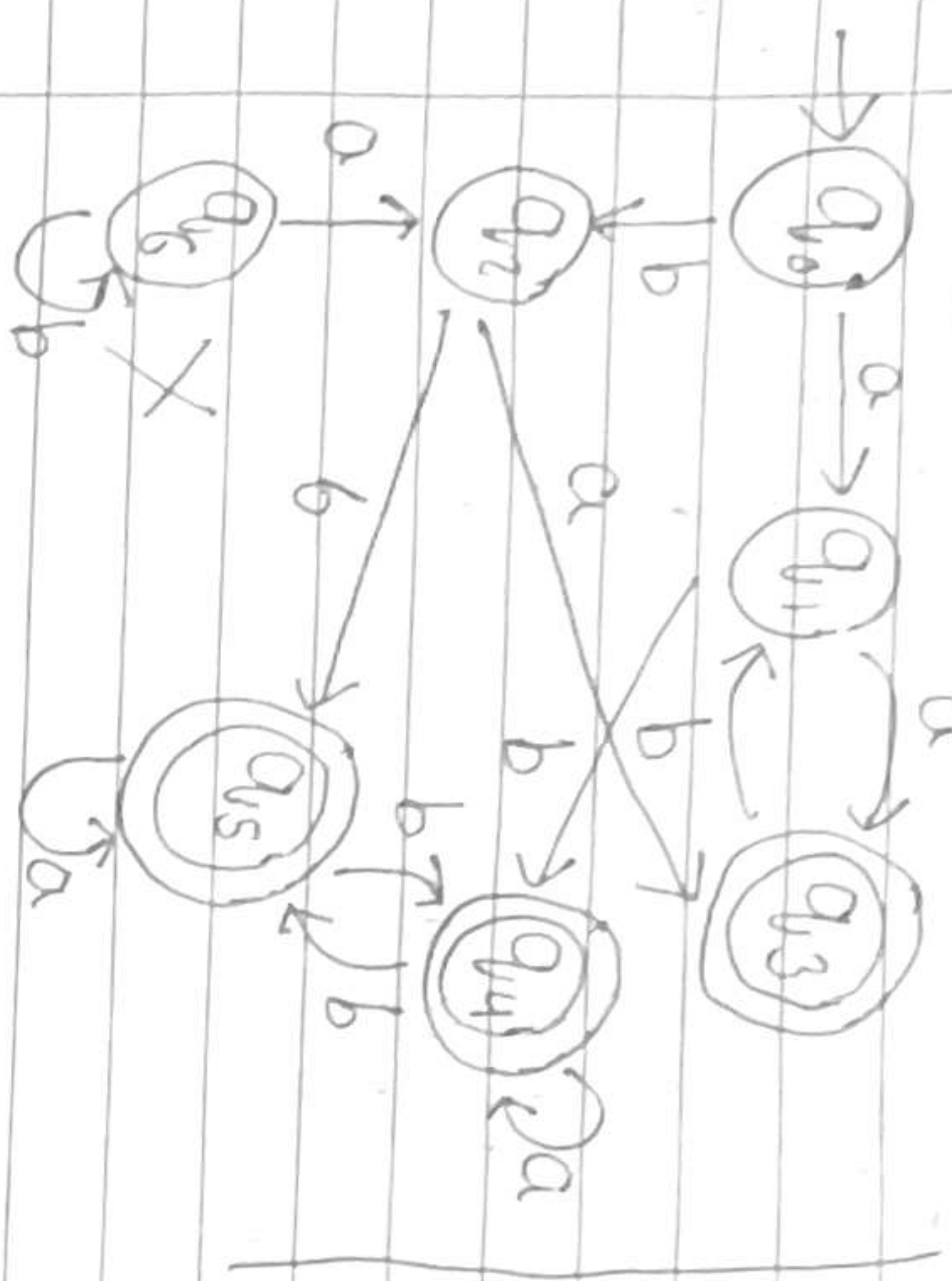
$a \in \{a, b\}^*$
 $a, b, c, aa, bb, cc, aab, bba, ccc$

$\{m \geq n, m > n, m < n\}$

Regular languages

finite
 $n=1000$
 $a^n, n \geq 0$

Minimization of DFA



q_0	q_1	q_2
q_2	q_3	q_5
q_4	q_5	q_5

Same group

	a	b
q_0	q_1	q_2
q_1	q_3	q_4
q_2	q_3	q_5
q_3	q_3	q_1
q_4	q_4	q_5
q_5	q_5	q_5

NFA $\rightarrow$ DFA
 $n \quad 2^n$

- 1) Remove unreachable states (Remove q_5)
- 2) Convert into equivalent classes

q_0, q_1 should be in same group
 q_2, q_3 in same group
 q_4, q_5 in same group

$q_0, q_1, q_2, q_3, q_4, q_5$ in same group

$\Pi = \{q_0\}$

$\Pi_0 = \{q_0, q_1, q_2\}$

$\{q_3, q_4, q_5\}$

$\{q_1, q_2\}$

$\{q_3\}$

$\{q_4, q_5\}$

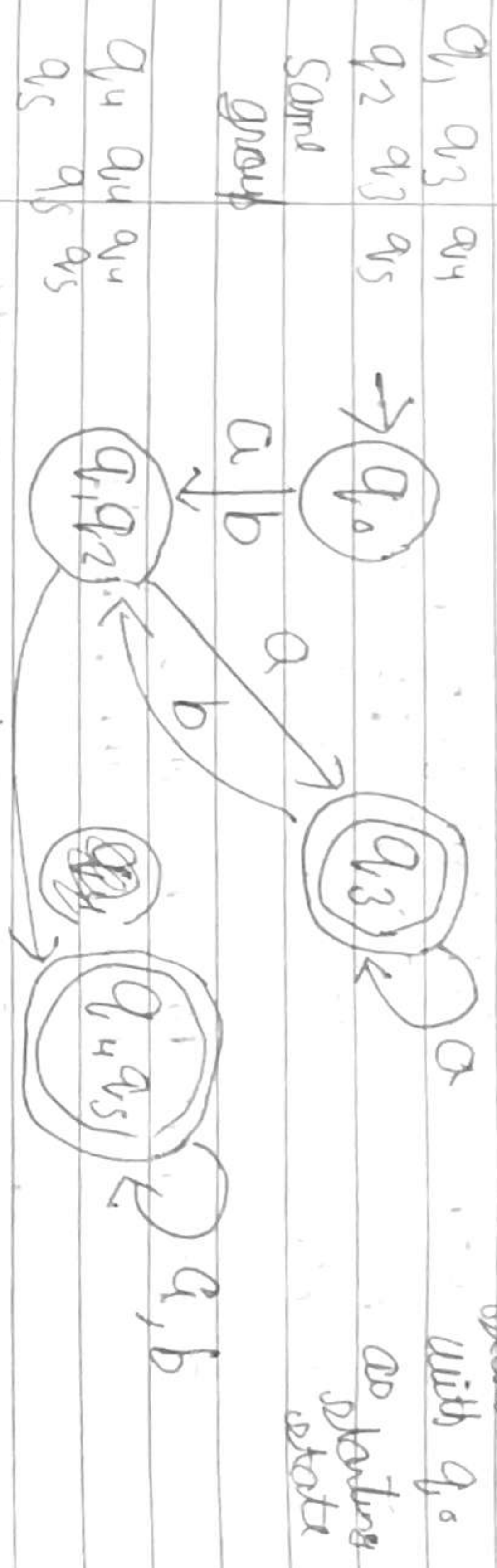
accepting / non accepting / final / non final / states

already minimized to one

$\Pi_2 = \{q_0\} \quad \{q_1, q_2\} \quad \{q_3\} \quad \{q_4, q_5\}$

New, we will compare with Π_1

Make q_0 as any other state with q_0



States with q_3, q_4, q_5 will be final state

language accepted by FA

Regular expression (Regular languages)

- Method to represent a language

- let 'R' be a regular expression over alphabet Σ if R is:

- (i) ϵ is regular expression denoting the set $\{\epsilon\}$
- (ii) ϕ is regular expression denoting the empty set $\{\}$
- (iii) For each symbol $a \in \Sigma$, a is regular expression denoting set $\{a\}$
- (iv) Union of 2 RE is also regular
- (v) Concatenation of two RE is also regular
- (vi) Kleene closure * of RE is also regular
- (vii) If R is regular, R^* is also Regular
- (viii) Nothing else, Repeat 1 to 7 recursively

$L = \{\epsilon, a, aa, aaa, \dots\}$

$R = \epsilon \quad L(R) = \{\epsilon\}$
 $R = \phi \quad L(R) = \{\}$
 $R = a \quad L(R) = \{a\}$

a^+
 min string = a
 a^*
 min string = ϵ

+ = UNION

Regular expression Example 1

- 1) No string $\{\}$ $RE = \phi$
- 2) length 0 $\{\epsilon\}$ $RE = \epsilon$
- 3) length 1 $\{a, b\}$ $RE = a + b$
- 4) length 2 $\{aa, ab, ba, bb\}$ $RE = (aa + ab + ba + bb)$
- 5) length 3 $\{aaa, aab, aba, abb, baa, bab, bba, bbb\}$ $RE = (aaa + aab + aba + abb + baa + bab + bba + bbb)$
- 6) Atmost 1 $\{a, b\}$ $RE = (a + b)$
- 7) Atmost 2 $\{\epsilon, a, b\}$ $RE = (1 + a + b)$

0. Not more than 2's and 1's
 $\{\epsilon, a, b, ab, ba, abb, bab, bba\}$

$$RE = (\epsilon + a + b + ab + ba + abb + bab + bba)$$

For Infinite Languages

Consider alphabet $\Sigma = \{a, b\}$

- 1) All strings having a single 'b' $\Rightarrow a^* b a^*$
- 2) All strings having atleast one 'b' $\Rightarrow (a + b)^* b (a + b)^*$
- 3) " " " bbb or substring $\Rightarrow (a + b)^* bbb (a + b)^*$
- 4) All strings end with 'ab' $\Rightarrow (a + b)^* ab$
- 5) " " " start with 'ba' $\Rightarrow ba (a + b)^*$
- 6) " " " beginning and end with 'a' $\Rightarrow a (a + b)^* a$
- 7) " " " containing 'a' $\Rightarrow a + a(a + b)^* a$
- 8) " " " starting and ending with different symbol $\Rightarrow (a(a + b)^* b + b(a + b)^* a)$

$$a(a + b)^* + b(a + b)^*$$

Finite always Regular

Q. Which 2 of the following 4 regular expressions are equivalent?

- (i) $(aa)^* (\epsilon + a)$
- (ii) $(aa)^*$
- (iii) 0^*
- (iv) $0 (00)^*$

- (a) (i) and (ii)
- (b) (ii) and (iii)
- (c) (i) and (iii)
- (d) (iii) and (iv)

Pumping lemma -

If L is an infinite language then there exists some positive integer 'n' (pumping length) such that any string $w \in L$ has length greater than or equal to 'n' i.e. $|w| \geq n$ then w can be divided into 3 parts $w = xyz$

It satisfies following conditions:
 1. for each $i \geq 0$, $xy^i z \in L$

- (i) $|y| > 0$ and
- (ii) $|xy| \leq n$
- (iii) $|y| \leq n$



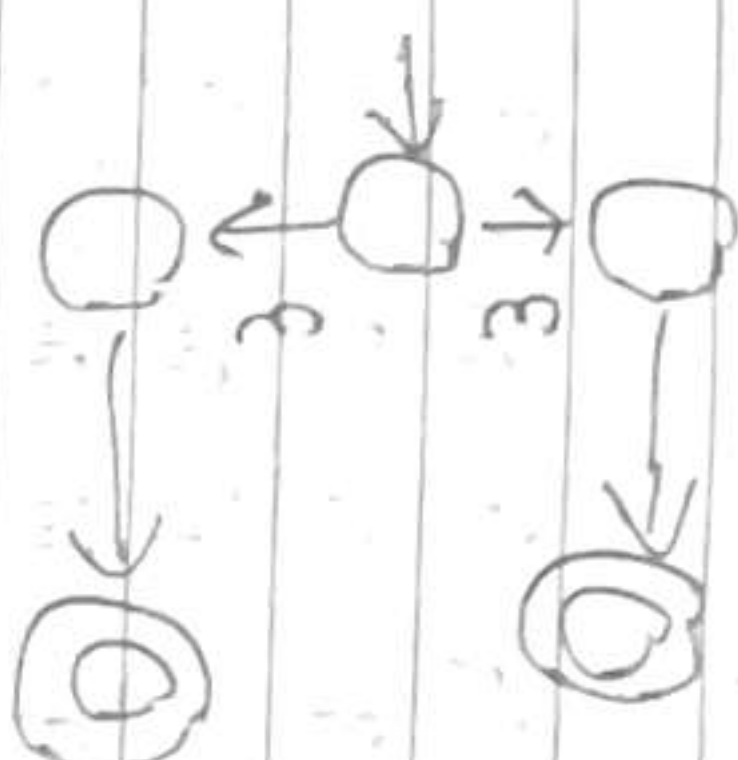
eg $a^n b^{2n}$ $n \geq 0$
 $aa bbb \in L$

$(y)^{i+1}$
 $aa bbb \notin L$

Non Regular (Decidable)
 a and b can't be alternate forms

Closure Properties of Regular Languages

- 1) Union ($L_1 \cup L_2$)
- 2) Concatenation ($L_1 L_2$)
- 3) Closure ($*$) L^*
- 4) Complement $\bar{L} = \Sigma^* - L$
- 5) Intersection $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$
- 6) Difference $L_1 - L_2 = L_1 \cap \bar{L}_2$
- 7) Reversal (L^R)
- 8) Homomorphism
- 9) Reverse homomorphism
- 10) Quotient Operation



Limitations of Pumping Lemma

- 1) Not all non context free languages violate the pumping lemma
- 2) Context free languages that do not satisfy the pumping lemma
- 3) Sensitivity to choice of decomposition
- 4) Pumping one non constructive
- 5) Does not prove context freeness
- 6) Limited to formal languages

Challenges

- 1) Proved by contradiction
- 2) Complexity by proof
- 3) Not a complete context characterisation
- 4) Difficult in finding the pumping length
- 5) Specific to context free languages

Limitations of Regular Languages

- 1) Cannot count matched pairs
- 2) Cannot handle nested structures
- 3) Cannot recognise palindromes
- 4) Cannot handle context sensitive languages

Properties of Regular Languages

- 1) Regular language to Regular Expressions
- 2) All strings over $\{0,1\}$ that end with 01
- 3) Regular Expression: $(0+1)^* 01$

- 2) All strings over $\{0,1\}$ that have at least one 'a'
- 3) All strings over $\{0,1\}$ that contain no consecutive 1's

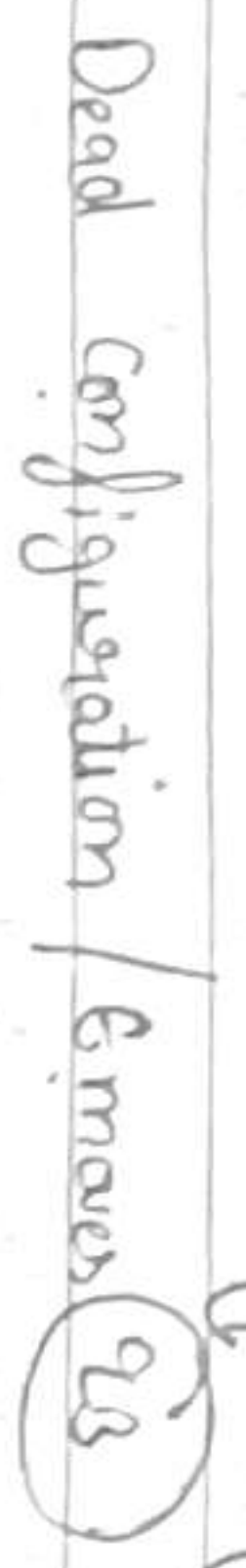
- 3) All strings over $\{0,1\}$ that contain no consecutive 1's
- Regular Expression for Finite Automata

Conversion of Regular expression to Finite Automata
Theorem: Every language defined by a regular expression, is also defined by a Finite Automata

$$0^* = \{ \epsilon, 0, 00, 000, \dots \}$$

$$0^*(10^*) = \{ 0, 01, 001, 0001, \dots \}$$

ab



90 a b



$f = f_0, 10, 100, 110, 1001, 0001,$


$$L = \{ \varepsilon, ab, aab, aaab, \dots \}$$

Let's take $aaabbb \in L$

Using	Take	
PLT	aa ad x	<u>aa</u> <u>ab</u> <u>bb</u>
	ab ay	a b c
	bb az	pump ab
		aa ab ac bb z d

let's assume $L = L(R)$ for regular expression R .

(i) Exactly one accepting state

- (iii) No outgoing edges at the accepting state

- 1) Create a starting state, say q_1 , and a final state, say q_2 .

But, if Γ is $(Q)^*$, Kleene's closure of another regular

will also be the final state, as in Fig 2.



Fig 1

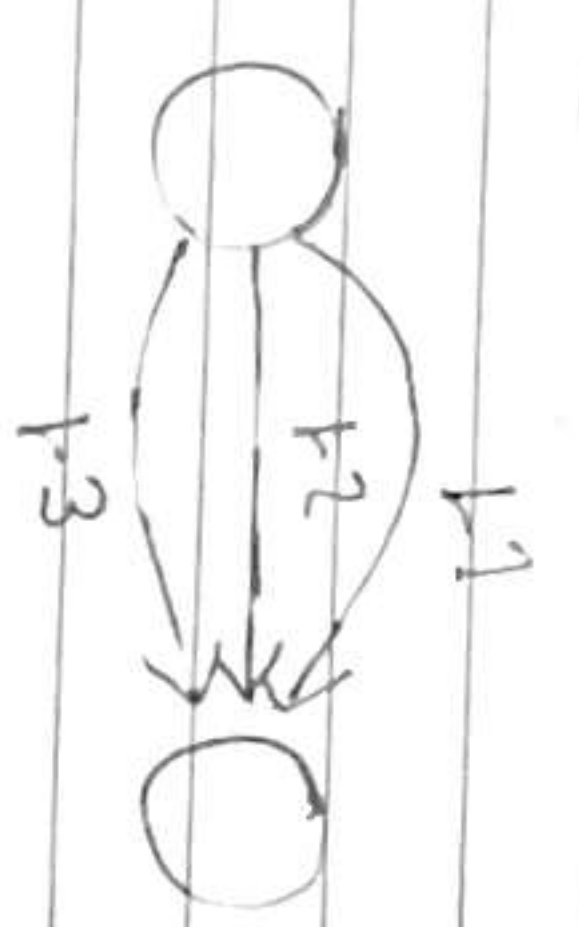
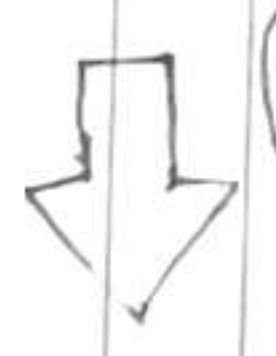
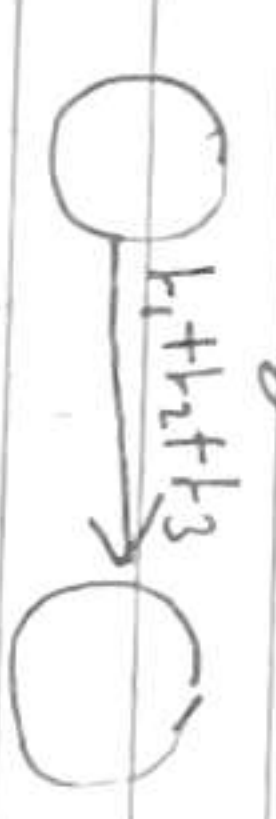
Fig 2

2) Repeat the following rules (rotate decomposition method)

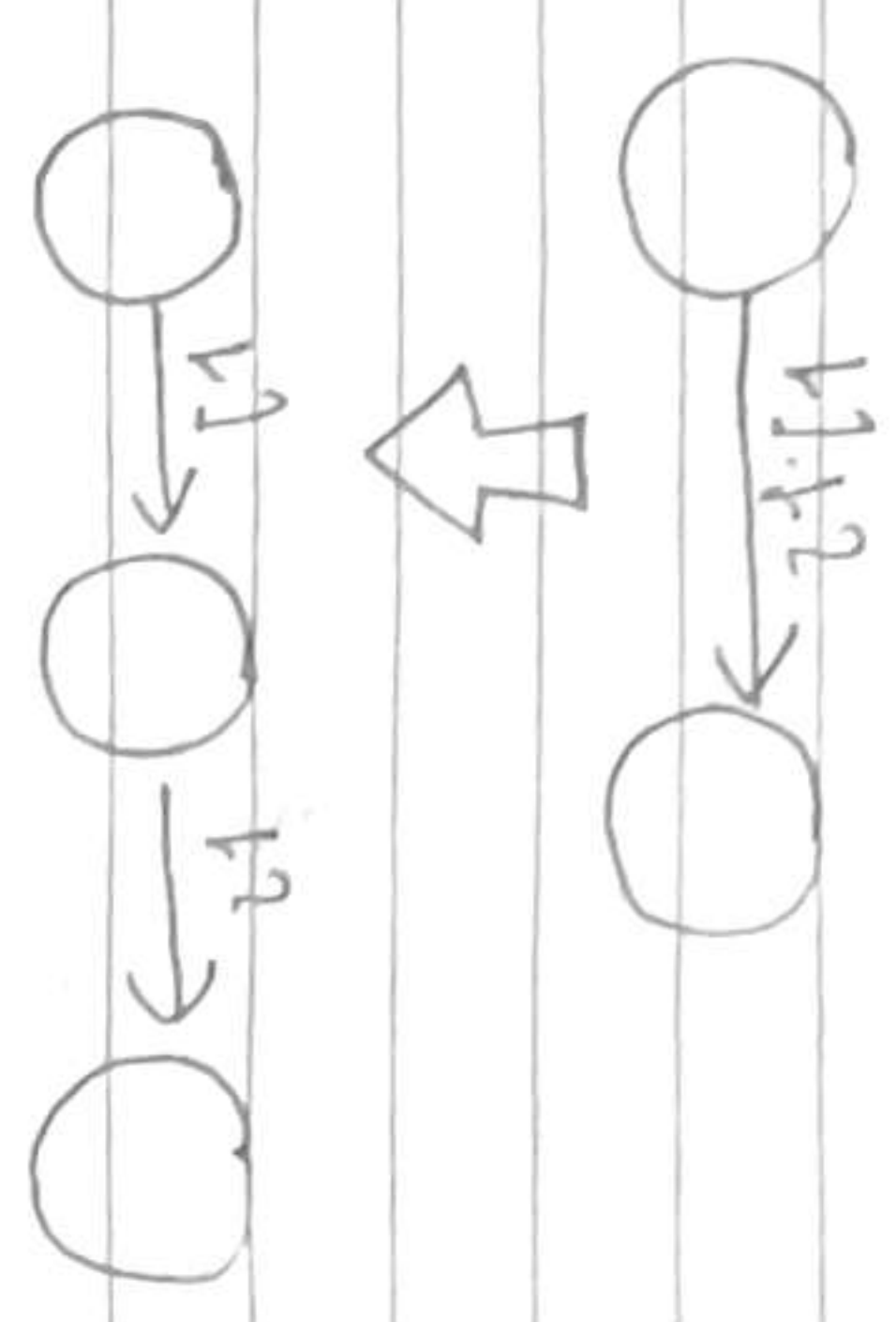
operation to link until no operation is left in the explicit

ad Union $<$ condensation $<$ Kleine's charge

parallel edges between the 2 states and 1 allowed

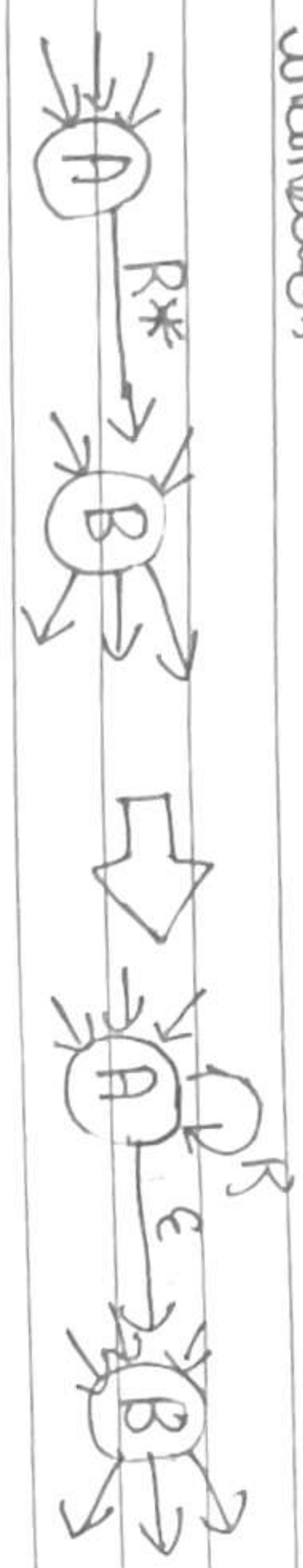


- Concatenation operation ('.') can be eliminated by introducing a new state between the states as follows.

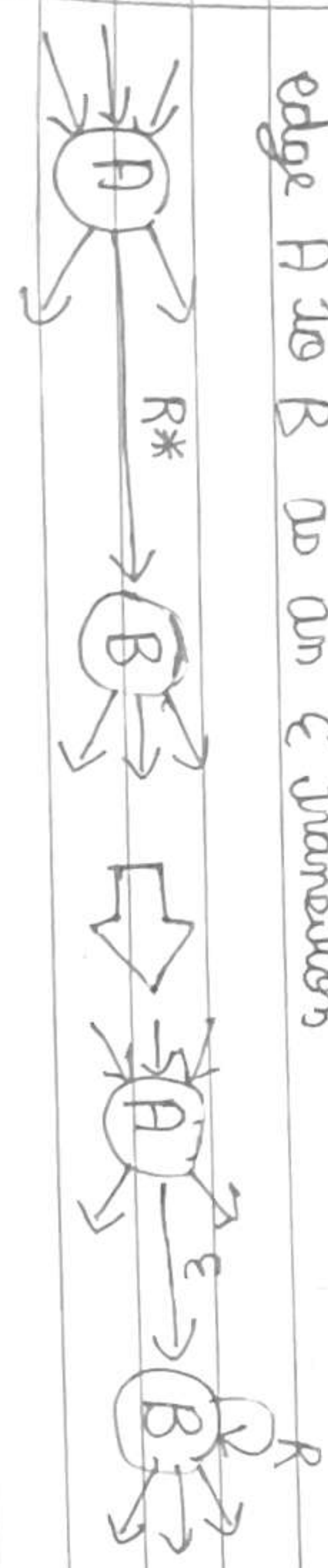


- Kleene's Closure (*) can be eliminated by introducing self loops on states labeled with the following conditions:

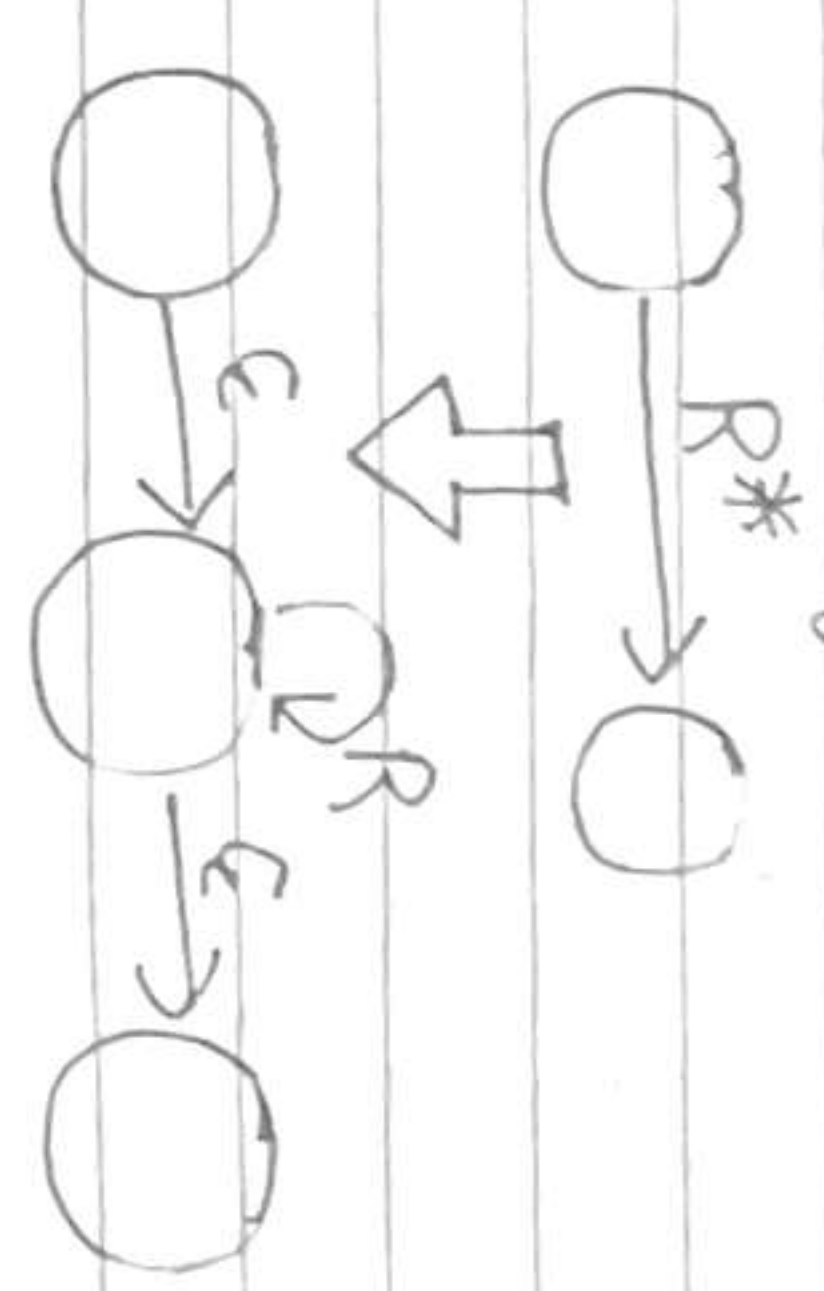
(i) If there is only one outgoing edge at the left most state, i.e. A in transition $A \rightarrow B$, then introduce self-loop on state A and label edge A to B as an ϵ transition.



(ii) Else if there is only one incoming edge at the right most state i.e. B in transition $A \rightarrow B$, then introduce self loop on state B and label edge A to B as an ϵ transition.



(iii) Else introduce a new state between 2 states having self loop labeled as the expression. The new state will have ϵ -transitions with the previous states as follows:

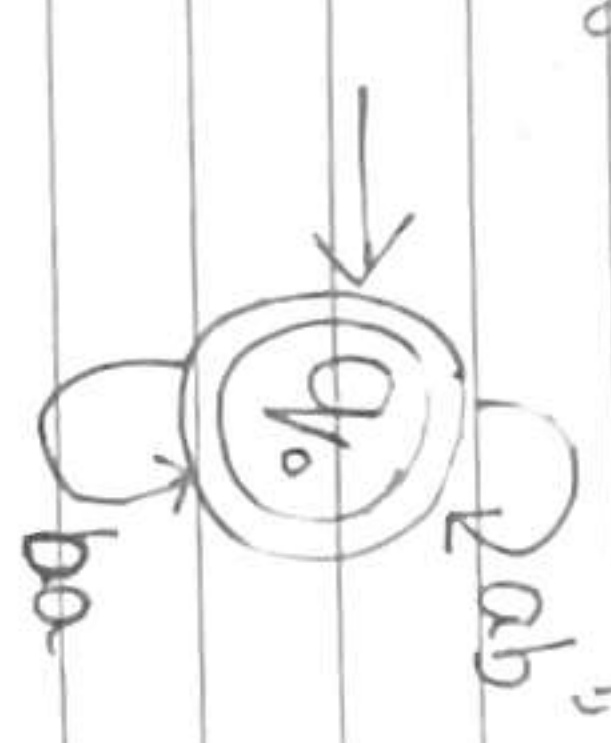


Q. Construct a Finite Automata for the regular expression, $R = (ab+ba)^*$

Ans. 1. As the given expression, R is of the form $(S)^*$, we will create a single initial state that will also be the final state, having self loop labeled $(ab+ba)$



2. As the least precedence operator in the expression is a union (+). So we will introduce parallel edges (parallel self loops here) for 'ab' and 'ba'



3. Now we have 2 labels with concatenation operator, so we remove them by one by one by introducing new states q_1 and q_2



Kleene's Theorem -

States that a language is regular iff it can be described by a regular expression. This theorem highlights the inherent connection between regular expressions & finite automata, affirming that any language recognizable by a finite automaton can be expressed by a regular expression, & vice versa. It also implies an equivalence with transition graphs.

2 main directions -

Theorem often understood through 2 consecutive directions:

1) RE to FA

Direction shows that any language defined by a regular expression can be represented by a FA.

This process typically involves converting regular expression into an equivalent NFA, using rules for operations like union, concatenation, and Kleene Star.

The resulting NFA can be converted into an equivalent DFA using methods such as subset construction algorithm.

2) FA to RE

Direction proves that any language recognized by a FA can be described by a regular expression.

The construction involves converting DFA into a Generalized NFA, where transitions are labeled with RE. States are then systematically eliminated from GNFA, updating transition labels,

until only a start symbol state and a single final state remain. The label on the transition between these 2 states becomes the equivalent RE.

* Arden's Theorem is a method often used to derive

* RE from FA by solving a system of equations based on state transitions.

Arden's Theorem -

Method used to derive a RE from FA.

It states that if P & Q are 2 regular expressions, and P does not contain the null string (ϵ), then the equation:

$$R = Q + RP$$

has a unique solution given by:

$$R = QP^*$$

This theorem provides a systematic way to find RE for a language accepted by a FA by setting up and solving a system of equations, where each state corresponds to a RE.

The solution $R = QP^*$ is derived by repeatedly substituting R into the equations, leading to $Q(\epsilon + P + P^2 + \dots)$ which simplifies to QP^* by the definition of Kleene Closure operation.

Regular Expression to DFA

$$RE = [ab + (b + aa)b^*a]$$

